Université de Sousse

École Nationale d'Ingénieurs de Sousse

Département d'Électronique Industrielle

Société d'accueil

AVIONAV

Mémoire de Projet de Fin d'Études

Présenté en vue de l'obtention du diplôme d'

Ingénieur en Génie Electronique Industrielle

Option : Systèmes Embarqués

Sujet :

« Smart Cockpit For Aircraft »

Réalisé par : **Oumayma FRADI**

Soutenu le **24 /06/2023** devant le jury :

Président	:	M. Anis BEN SLIMANE, ENISo
Rapporteur	:	Mme. Intissar MOUSSA ESSOUSSI, ENISo
Encadrant académique	:	M. Adel BOUALLEGUE, ENISo
Encadrant Professionnel	:	M. Foued EL KAMEL, AVIONAV

©Oumayma FRADI 2023

Résumé

La somnolence des pilotes est un risque majeur d'accidents aériens, car elle affecte leur vigilance, leur temps de réaction et leurs performances cognitives. Cela peut entraîner des erreurs de jugement et compromettre la sécurité lors de la conduite d'avion.

Afin de prévenir les accidents aériens liés à la somnolence des pilotes, j'ai eu l'opportunité de concevoir une application intelligente destinée à être intégrée dans le poste de pilotage de l'avion, dans le cadre de mon projet de fin d'études.

Cette application utilise des technologies avancées de vision par ordinateur pour détecter les signes de somnolence tels que la fermeture des yeux ou les bâillements. Lorsque ces signes sont détectés, l'application alerte le pilote pour prendre des mesures immédiates.

Notre application est équipée d'une fonctionnalité d'urgence. Elle permet à l'utilisateur d'envoyer rapidement des messages d'alerte, de passer des appels et de partager instantanément la localisation en cas de situation critique. Cette fonctionnalité garantit l'obtention d'une assistance rapide en cas d'urgence.

Mots clés **:** Apprentissage automatique - Vision Par Ordinateur - Machine Learning Kit - Google Play Services - Suivi de visage - Détection en temps réel - Android Studio – Java - Interface utilisateur – SOS - Localisation.

Abstract

Pilot drowsiness is a major risk factor for aviation accidents as it affects their vigilance, reaction time, and cognitive performance. This can lead to judgment errors and compromise safety during aircraft operation.

To prevent aviation accidents related to pilot drowsiness, I had the opportunity to design an intelligent application intended to be integrated into the aircraft's cockpit as part of a final-year project.

This application utilizes advanced computer vision technologies to detect signs of drowsiness such as eye closure or yawning. When these signs are detected, the application alerts the pilot to take immediate action.

It is equipped with an emergency function. It allows the user to send alert messages, make calls and share the location in case of a critical situation. This feature ensures an assistance in case of emergency.

Keywords: Machine Learning - Computer Vision - Machine Learning KIT - Google Play Services - Face Tracking - Real time detection - Android Studio – Java - User Interface - Emergency - Location.

« Si vous mettez votre cœur dans la réalisation de vos projets, si la passion vous dévore et que rien ne vous arrête, succès il y aura. »

-Mathieu Thomas-

Dédicaces

Je dédie ce travail du fond du cœur,

À mes chers parents TAREK et NAJOUA

Pour leur patience illimitée et leur encouragement contenu. J'espère qu'ils trouvent ici, la récompense de leurs sacrifices.
Aucune dédicace ne saurait être assez éloquente pour exprimer l'amour et le respect que j'ai toujours pour vous.

À mes adorables sœurs ONES et KHAOULA

Pour tous les moments de joie que nous partageons ensemble. Pour votre amour, votre compréhension et votre patience.

À mes chers frères MOHAMED et MOHAMED GHALI

Pour tout l'amour qu'ils me portent. Pour leurs encouragements qu'ils m'ont apportés.

À tous mes amis qui m'ont soutenu.
À tous ceux que j'aime.

FRADI Oumayma …

Remerciements

En tout premier lieu, je remercie **ALLAH**, de m'avoir donné l'audace et la souffle pour accomplir avec succès mon parcours.

Je souhaite exprimer ma sincère gratitude envers mon encadrant académique **M. ADEL BOUALLEGUE** en lui témoignant toute ma reconnaissance pour ses précieux conseils et la confiance qu'il m'a témoignée. Je tiens également à exprimer mon profond respect envers lui.

Je tiens à exprimer ma gratitude la plus sincère envers mon encadrante académique **Mme SAFA AMEUR** pour sa disponibilité continue et ses précieux conseils. Je suis extrêmement reconnaissante pour votre soutien inestimable tout au long de cette période.

Je tiens à remercier mon encadrant professionnel **M. FOUED EL KAMEL** pour l'accueil chaleureux au sein de la société et pour le bon conseil.

J'exprime toute ma reconnaissance à **M. ANIS BEN SLIMANE** pour avoir bien voulu accepter de présider le jury de ce mémoire.

J'espère que **Mme INTISSAR MOUSSA ESSOUSSI** trouve ici l'expression de mes vifs remerciements pour avoir bien voulu juger ce travail.

Je remercie **nos meilleurs professeurs de L'ENISO** d'avoir enrichi nos connaissances et de nous avoir montré les clés du succès**.**

-Que dieu vous comble de sa grâce chaque jour de votre vie-

Table des matières

Liste des tableaux ..... *IX*

Liste des figures ..... *X*

Liste des abréviations ..... *XII*

Introduction Générale *1*

Chapitre 1 : Cadre générale du projet *3*

1 Introduction **3**

2 Présentation de la société **3**

2.1 Historique 4

2.2 Organigramme de la société 4

2.3 Identité 4

2.4 Typologie de production 5

2.5 Architecture du local 5

2.6 Produits de société 6

3 Présentation du projet **7**

3.1 Contexte général du projet 7

3.2 Problématique 7

3.3 Objectifs 9

4 Etude de l'existant **9**

4.1 Systèmes à base des capteurs 10

4.2 Les systèmes d'apprentissage automatique 11

4.3 Etude comparatif entre les solutions existantes 12

5 Solution proposée **13**

5.1 Présentation 13

5.2 Caractéristiques 13

5.3 Originalité 14

6 Planning du projet **15**

7 Conclusion **15**

Chapitre 2 : Analyse et Spécification *16*

1 Introduction **16**

2 Identification des besoins **16**

2.1 Besoins fonctionnels 16

2.2 Besoins non fonctionnels 17

3 Analyse fonctionnelle **18**

3.1 Diagramme de Bête à cornes 18

3.2 Diagramme de Pieuvre 18

3.3 Diagramme SWOT 19

4 Modélisation conceptuelle 20
4.1 Le Langage de Modélisation Unifié 20
4.2 Diagramme de cas d'utilisation 20
4.3 Diagramme de séquences 23
4.4 Diagramme d'activité 24
5 Conclusion 24
Chapitre 3 : Développement et Implémentation 25
1 Introduction 25
2 Environnement matériel 25
3 Environnement logiciel 26
3.1 Android studio 26
3.1.1 Présentation 26
3.1.2 Version 27
3.1.3 Architecture 28
3.1.4 Structure du projet 29
3.2 Android JDK 30
3.3 Android SDK 30
4 Choix du Langage 31
4.1 Java 31
4.2 XML 31
5 Framework et technologie 32
5.1 ML Kit 32
5.2 ML KIT Face Detection API 32
5.3 Google Play Services 33
6 Implémentation 33
6.1 Configuration 33
6.1.1 Chemins d'installation de modèle 34
6.1.2 Installation de modèle 35
6.2 Méthodologie 35
6.2.1 Détection de somnolence 36
6.2.2 Gestion des cas d'urgence 38
6.3 Installation 39
7 Conclusion 40
Chapitre 4 : Test et Evaluation 41
1 Introduction 41
2 Résultat 41
2.1 Structure hiérarchique 41
2.2 Interfaces 42
2.2.1 Logo et Dénomination 42
2.2.2 Start Page 42
2.2.3 Home Page 43

2.2.4 Main Activity Page 43
2.2.5 Menu Page 45
2.2.6 Location Page 46
2.2.7 About App Page 46
2.2.8 Contact Us Page 47
3 Test 47
3.1 Précision 47
3.1.1 Test avec luminosité variée 48
3.1.2 Test d'inclinaison de caméra 48
3.1.3 Test divers 49
3.2 Performance 50
3.2.1 Profileur 50
3.2.2 Etude des performances 51
4 Conclusion 53
Conclusion générale 54
Référence 56

Liste des tableaux

Tableau 1 : Tableau d'identité de la société « AVIONAV » 4
Tableau 2 : Tableau comparatif entre les solutions existantes..... 12
Tableau 3 : Etude comparative 14
Tableau 4 : Description des fonctions principales et contraintes du système 19
Tableau 5 : Description de cas d'utilisation « Lancer l'application » 21
Tableau 6 : Description de cas d'utilisation « Lancer la détection de somnolence » 21
Tableau 7 : Description de cas d'utilisation « Envoyer la localisation » 22
Tableau 8 : Les fonctionnalités d'Android studio..... 26
Tableau 9 : Les versions d'Android 27
Tableau 10 : Les différences entre options d'installation [20]..... 34
Tableau 11 : Les caractéristique des options d'installations [21] 34
Tableau 12 : Les classes d'affichage..... 36
Tableau 13 : Classes de détection de visage..... 37
Tableau 14 : Conditions & Evènement..... 37
Tableau 15 : Liste des permissions..... 38

Liste des figures

Figure 1 : LOGO de « AVIONAV » [2] 3
Figure 2 : Organigramme de la société 4
Figure 3 : Sous-sol 5
Figure 4 : Rez de chaussée 5
Figure 5 : 1er Etage 5
Figure 6 : Rallye full Carbon [3] 6
Figure 7 : Storm Century SLS [3] 6
Figure 8 : avion Fury [3] 6
Figure 9 : Accidents attribués à la somnolence 7
Figure 10 : Variation des pourcentages d'erreurs entre la somnolence et l'état active [6] 8
Figure 11 : Les causes et les effets de fatigue [4] 8
Figure 12 : Les solutions existantes pour détecter la somnolence chez les pilotes 9
Figure 13 : capteur EEG [7] 10
Figure 14 : capteur EMG [8] 10
Figure 15 : capteur de mouvement [9] 10
Figure 16 : Les étapes d'apprentissage automatique 11
Figure 17 : Schéma synoptique de la solution 13
Figure 18 : Diagramme du GANTT 15
Figure 19 : Diagramme de Bête à cornes 18
Figure 20 : Diagramme de pieuvres 18
Figure 21 : Diagramme SWOT 19
Figure 22 : Diagramme de cas d'utilisation globale 20
Figure 23 : Diagramme de séquences 23
Figure 24 : Diagramme d'activité 24
Figure 25 : Evolution d'Android [12] 27
Figure 26 : Architecture Android 28
Figure 27 : Fichiers de projet de la vue Android [14] 29
Figure 28 : Version de JDK 30
Figure 29 : Fonctionnalités d'Android SDK 30
Figure 30 : Logo de Java [15] 31
Figure 31 : Logo XML [16] 31
Figure 32 : Mises à jour d'application [19] 33
Figure 33 : Déclaration de modèle [21] 35
Figure 34 : Dépendance des bibliothèques ML kit [21] 35
Figure 35 : Architecture de système globale 35
Figure 36 : Boite de dialogue 39
Figure 37 : Arborescence d'application 41
Figure 38 : Logo d'application 42
Figure 39 : Start Page 42
Figure 40 : Home Page 43
Figure 41 : Situation normale 44
Figure 42 : Situation de somnolence 44
Figure 43 : Situation de bâillement 45

Figure 44 : Menu Page 45
Figure 45 : Location page 46
Figure 46 : About app page 46
Figure 47 : Contact Us Page 47
Figure 48 : Test de précision avec différents types de luminosité..... 48
Figure 49 : Test de précision avec des inclinaisons de caméra..... 48
Figure 50 : Test de précision -cas 1 49
Figure 51 : Test de précision -Cas 2..... 49
Figure 52 : Vue chronologique des activités et fragments 51
Figure 53 : Vue des performances d'activité principale..... 52

Liste des abréviations

OACI	:	Organisation de l'aviation civile internationale
NSF	:	National Sleep Foundation
IA	:	l'intelligence artificielle
CNN	:	Réseaux de neurones convolutionnels
JDK	:	Java Development Kit
SDK	:	Software Development Kit
XML	:	Acronyme eXtensible Markup Language
API	:	Application Programming Interface
ML KIT	:	Machine Learning Kit

Introduction Générale

Au fil des années, le domaine de l'aéronautique a connu une évolution rapide grâce aux innovations technologiques, la conception et la fabrication des avions. Cependant, malgré ces progrès réalisés, les accidents d'aviation continuent de se produire d'une manière récurrente.

L'un des facteurs principaux de ces accidents est la fatigue des pilotes. C'est un sujet de recherche majeur dans l'aviation depuis de nombreuses années et les études ont démontré que la somnolence est un facteur causal dans de nombreux accidents et incidents aériens.

*D*e surcroît, la somnolence des pilotes peut être influencée par de nombreux facteurs, tels que les horaires de travail irréguliers, les longues heures de vol, les décalages horaires, le stress et les conditions environnementales. Ces éléments peuvent entraîner une privation de sommeil, perturber les cycles de sommeil naturels et accumuler de la fatigue chez les pilotes.

Il est donc essentiel de sensibiliser et prendre des mesures pour réduire les risques liés à la fatigue des pilotes. Face à cet enjeu, nous avons réfléchi à une solution innovante : Une application de surveillance de la somnolence en temps réel, intégrée d'une manière simple et non intrusive dans le poste de pilotage. Elle offre une solution intelligente pour atténuer les risques liés à la fatigue des pilotes et elle permet de gérer efficacement les situations d'urgence.

En vue de rendre compte d'une manière fidèle et analytique du période passé au sein de la société **AVIONAV**, j'ai élaboré ce rapport qui sera divisé en quatre chapitres :

- **Le premier chapitre** sera structuré en deux parties principales :
- La première partie sera consacrée à la présentation de l'entreprise, mettant en évidence son contexte dans le domaine de l'aéronautique.
- Dans la deuxième partie, nous aborderons l'objectif du stage et nous présenterons le cadre général de notre projet. Nous introduirons ensuite l'étude et la critique des solutions existantes, ainsi que la solution que nous proposons.
- **Le deuxième chapitre** sera consacré à une analyse des besoins. D'une part nous examinerons en détail les besoins fonctionnels et non fonctionnels. Et d'autre part nous présenterons les besoins sous forme de diagrammes, afin de faciliter une meilleure compréhension.

- **Le troisième chapitre** se focalisera sur les outils utiles pour le développement de notre application. D'abord Nous présenterons les environnements matériels et logiciels. Ensuite nous aborderons également les langages de programmation que nous avons adoptés, ainsi que les Frameworks et les technologies utilisés pour faciliter le processus de développement. Enfin, nous décrirons en détail la procédure d'implémentation de notre application, en mettant l'accent sur les étapes clés.

- **Le quatrième chapitre** est dédié aux tests et à l'évaluation de notre application, nous débuterons par une analyse approfondie des interfaces. Ensuite, nous effectuerons une série de tests afin d'évaluer la précision et le bon fonctionnement de l'application dans divers scénarios. Enfin, nous examinerons les performances de l'application pour analyser les résultats.

En conclusion, ce projet se termine avec une synthèse des principaux points développés et une exploration des perspectives envisageables pour améliorer cette solution.

Chapitre 1 : Cadre générale du projet

1 Introduction

Dans ce chapitre nous mettrons en évidence le contexte global dans lequel le projet s'est déroulé. Tout d'abord, nous introduisons l'entreprise d'accueil "AVIONAV", en décrivant sa structure organisationnelle, son domaine d'activité, ses produits et sa typologie de production. Ensuite, nous abordons le contexte du projet, en incluant la problématique, les objectifs ainsi que les solutions existantes, puis nous présentons notre propre solution, en proposant une approche innovante et adaptée.

2 Présentation de la société

AVIONAV [1] est une entreprise tunisienne basée à Borjine, dans le gouvernorat de Sousse. Elle est spécialisée dans la construction des avions légers. Grâce à son développement technologique avancé et à l'absence de concurrents sur le marché tunisien, « AVIONAV » a réussi à établir sa position dominante en tant qu'acteur clé de l'industrie aéronautique.

L'entreprise possède un atelier spécialisé dans divers domaines tels que les composites de haute performance, la tôlerie fine et industrielle, l'usinage de précision, la peinture et le montage. Ces compétences lui permettent de produire des avions de haute qualité et performants.

« AVIONAV » s'engage constamment dans des efforts soutenus pour fournir des solutions aériennes de pointe. Cela démontre son engagement envers l'innovation et son désir de rester à la pointe de la technologie dans le secteur de la construction aéronautique.

Figure 1 : LOGO de « AVIONAV » *[2]*

2.1 Historique

L'évolution de la société « AVIONAV » est retracée à travers les étapes clés suivantes :

2007 : Fondation de l'entreprise STORM AIRCRAFT par un groupe d'entrepreneurs italiens.
2011 : Rachat de l'entreprise par les frères EL Kamel.
2016 : Engagement dans la fabrication d'un aéronef en partenariat avec Evada Aircraft.
2017 : Exportation de plusieurs avions légers vers des pays d'Europe
2023 : Lancement de projet d'assemblage des hélicoptères.

2.2 Organigramme de la société

La société « AVIONAV » se représente à travers une structure hiérarchique et des relations entre les différents départements et postes au sein de la société. La figure 2 illustre l'organigramme de l'entreprise :

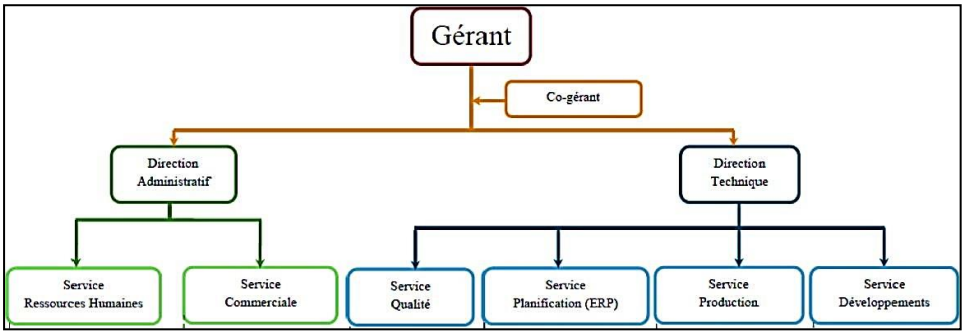


Figure 2 : Organigramme de la société

2.3 Identité

La société d'accueil est décrite par un ensemble des détails d'identité dans le tableau 1.

Nom	Domaine d'activité	Produits	Site Internet	Manager
Avionav	Industrie aéronautique	Avions légers - Hélicopter	https://avionav.net/	Foued EL KAMEL

Tableau 1 : Tableau d'identité de la société « AVIONAV »

2.4 Typologie de production

Pour décrire la typologie de production chez « AVIONAV », voici les éléments clés à prendre en compte :

- Le volume de production annuel est d'environ 10 avions.
- Le flux de production est discontinu en raison de la grande diversité d'articles et de variantes à produire. Les ateliers fonctionnels sont en mesure de traiter tous les types des produits en utilisant les outils appropriés et le flux des produits dépend de la séquence des tâches à effectuer.
- Chaque client commande un article spécifique.

La production est généralement effectuée sur commande. Cependant, même en période de crise ou de baisse des commandes, l'atelier continue de fonctionner et produit des petites pièces.

2.5 Architecture du local

Le local est réparti comme suit :

- Un rez-de-chaussée d' une superficie d'environ 430 m², composé d'un atelier dédié à l'assemblage de deux types d'avions. Il comprend également une salle d'exposition où les clients peuvent découvrir les différents produits proposés par l'entreprise.
- Un premier étage utilisé spécifiquement pour le moulage et le stockage des pièces et des produits composites, ainsi que pour le stockage des produits chimiques tels que la résine, le catalyseur et le durcisseur.
- Un sous-sol réservé à l'espace de stockage.

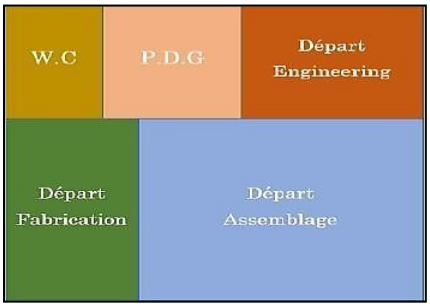


Figure 3 : 1er Etage



Figure 3 : Rez de chaussée



Figure 5 : Sous-sol

2.6 Produits de société

- **Rallye full Carbon** : Le Rallye est le dernier modèle de conception de pointe des avions légers spéciaux. C'est le fruit de nombreuses années de recherche et développement approfondis.



Figure 4 : Rallye full Carbon [3]

- **Storm Century Salsa** : est un avion en métal à aile basse offrant des performances, un cockpit spacieux, une visibilité exceptionnelle et une grande autonomie de vol grâce à ces réservoirs de carburant de grande capacité et sa conception de réduction de la traînée.



Figure 5 : Storm Century SLS [3]

- **Avion Fury** : Le Fury est un avion en aluminium qui présente des similitudes avec les deux avions précédents, mais il se distingue par sa structure de fuselage et son empennage.



Figure 6 : avion Fury [3]

3 Présentation du projet

Dans cette section nous situons le projet dans son contexte global, nous identifions la problématique et nous établissons les objectifs à atteindre.

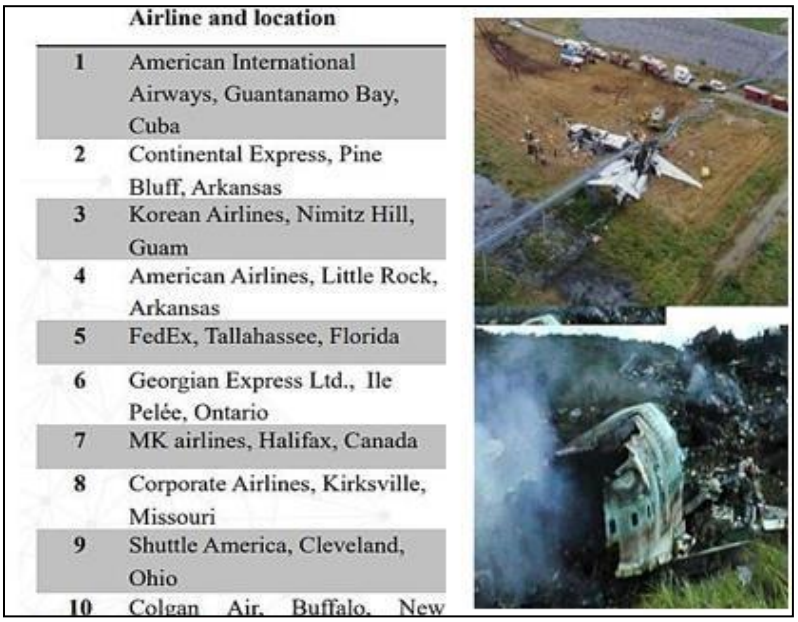
3.1 Contexte général du projet

Dans le domaine de l'aéronautique, la sécurité des avions est une préoccupation primordiale, l'état de vigilance des pilotes joue un rôle essentiel dans la garantie de vols sûrs. La fatigue, et plus particulièrement la somnolence, constitue la principale menace pour la sécurité.

La somnolence est un état de sommeil qui se caractérise par une envie irrésistible de dormir alors que la fatigue est un état de lassitude physique et mentale causée par un manque de repos adéquat ou une surcharge de travail. Ces deux termes sont étroitement liés car la fatigue excessive peut conduire à la somnolence.

3.2 Problématique

D'après les données de l'Organisation de l'aviation civile internationale, la somnolence a été identifiée comme un facteur contributif dans environ 15 à 20 % des accidents d'avion. De plus, une étude menée par la National Sleep Foundation a révélé que 20 % des pilotes ont commis une erreur majeure en raison de la somnolence [4]. La figure 7 représente quelques Accidents attribués à la somnolence.



	Airline and location
1	American International Airways, Guantanamo Bay, Cuba
2	Continental Express, Pine Bluff, Arkansas
3	Korean Airlines, Nimitz Hill, Guam
4	American Airlines, Little Rock, Arkansas
5	FedEx, Tallahassee, Florida
6	Georgian Express Ltd., Ile Pelée, Ontario
7	MK airlines, Halifax, Canada
8	Corporate Airlines, Kirksville, Missouri
9	Shuttle America, Cleveland, Ohio
10	Colgan Air, Buffalo, New

Figure 7 : Accidents attribués à la somnolence

La somnolence de pilote [5] présente des risques critiques pour la sécurité aérienne qui peuvent se résumer comme suit :

- Diminution de la vigilance : la somnolence réduit la capacité des pilotes à rester attentifs .
- Altération des fonctions cognitives : La somnolence affecte les fonctions cognitives telles que la mémoire, l'attention, la concentration et la perception.
- Réduction de temps de réaction : La somnolence ralentit le temps de réaction des pilotes, ce qui peut causer des retards dans les réponses aux événements inattendus. La figure 8 représente une variation des pourcentages d'erreurs entre la somnolence et l'état active.

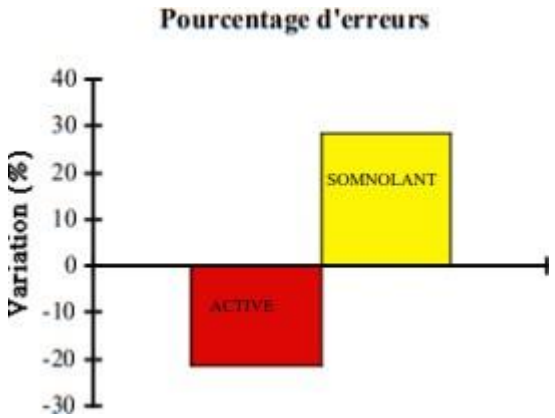


Figure 8 : Variation des pourcentages d'erreurs entre la somnolence et l'état active *[6]*

Les causes et les effets de fatigue cités dans la figure 9 soulignent l'importance de mettre en place des mesures pour prévenir la somnolence chez les pilotes, afin d'assurer la sécurité des vols.

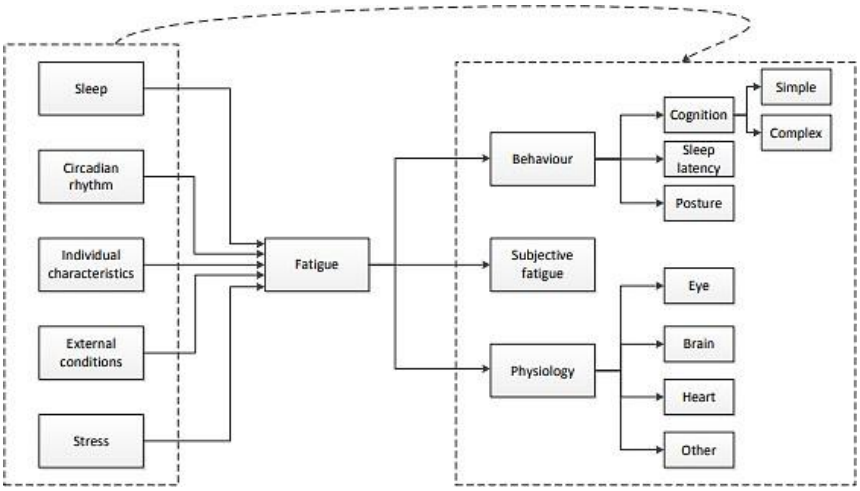


Figure 9: Les causes et les effets de fatigue [4]

3.3 Objectifs

L'objectif de notre projet vise à développer une application embarquée dans un appareil sous le système d'exploitation Android. L'application doit offrir les fonctionnalités suivantes :

- La détection de la somnolence.
- La gestion des cas d'urgence.
- La gestion de la localisation.

Cette application vise à créer une solution complète qui permettra aux utilisateurs de surveiller leur niveau de vigilance dans le but de :

- Prévenir les accidents.
- Améliorer la sécurité aérienne.
- Optimiser les décisions opérationnelles.

4 Etude de l'existant

Dans cette section, nous analysons les solutions déjà disponibles pour détecter la somnolence des pilotes, ainsi que leurs limites. Cette étape est cruciale pour identifier les différents besoins et exigences à prendre en compte lors de la phase de conception.
Il existe plusieurs systèmes de détection de somnolence spécialement conçus pour les pilotes. Ces systèmes visent à surveiller l'état de vigilance des pilotes pendant les vols afin de prévenir les risques liés à la somnolence et de garantir la sécurité des vols. La figure 10 présente quelques exemples des solutions :

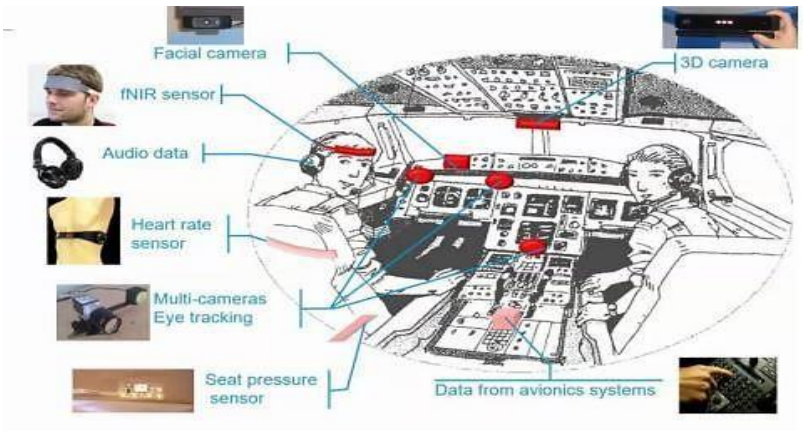


Figure 10 : Les solutions existantes pour détecter la somnolence chez les pilotes

Étant donné la diversité des solutions de détection de somnolence disponibles, nous avons pris la décision de diviser les systèmes en deux catégories :

4.1 Systèmes à base des capteurs

Les systèmes de détection de sommeil basés sur des capteurs utilisent des dispositifs sensoriels pour collecter des données. Voici quelques exemples de capteurs couramment utilisés :

- ✓ **Capteurs d'électroencéphalographie (EEG) :** Ces capteurs mesurent les activités électriques du cerveau

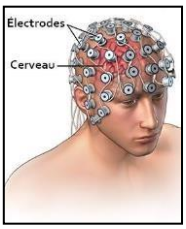


Figure 11 : capteur EEG *[7]*

- ✓ **Capteurs d'électromyographie (EMG)** : Ces capteurs enregistrent l'activité électrique des muscles, ce qui permet de détecter les mouvements involontaires.

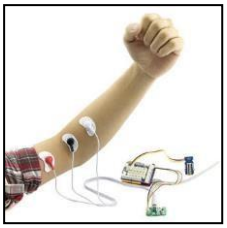


Figure 12 : capteur EMG *[8]*

- ✓ **Capteurs de mouvement** : Les capteurs d'accéléromètre ou de gyroscope peuvent êtreutilisés pour mesurer les mouvements corporels.

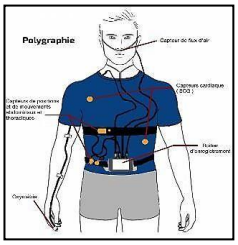


Figure 13 : capteur de mouvement *[9]*

Ces capteurs présentent certains inconvénients, notamment :

- Inconfort pour l'utilisateur : Les capteurs utilisés sont rigides.
- Limitations de la portabilité : Certains systèmes nécessitent des dispositifs encombrants.
- Limitation de précision et de performance : Les capteurs peuvent produire des résultats incorrects.

4.2 Les systèmes d'apprentissage automatique

Les systèmes de détection de somnolence basés sur des algorithmes d'apprentissage automatique utilisent des modèles et des techniques d'intelligence artificielle pour analyser et interpréter les données.

Ils sont développés par les étapes illustrés dans la figure 14 et ordonnancés comme suit :

1. Collecte des données : Des enregistrements sont collectés par des capteurs. Il est important d'avoir un ensemble de données diversifiés pour entraîner le modèle de détection.

2. Prétraitement des données : Les données collectées doivent être prétraitées pour éliminer les interférences indésirables et améliorer la qualité des données. Cela peut inclure des étapes telles que le filtrage et l'élimination des mouvements des bruits.

3. Extraction des caractéristiques : Des caractéristiques pertinentes sont extraites des données prétraitées. Cela peut inclure des caractéristiques temporelles, spectrales ou statistiques.

4. Entraînement du modèle : Les caractéristiques extraites sont utilisées pour entraîner les modèles d'apprentissage automatique, tels que les réseaux de neurones convolutionnels ou les arbres de décision.

5. Validation et évaluation : Le modèle entraîné est évalué sur un ensemble de données de validation pour estimer ses performances. Des métriques telles que la précision, le rappel et la spécificité, sont utilisés pour évaluer la précision de la détection du sommeil

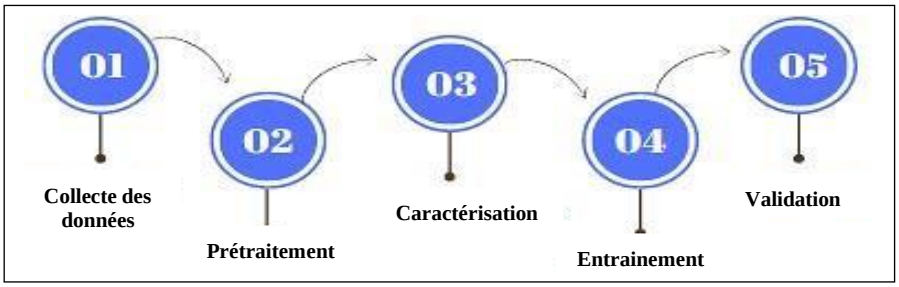


Figure 14 : Les étapes d'apprentissage automatique

Les principaux inconvénients des systèmes de détection de somnolence basés sur des algorithmes d'apprentissage automatique sont :

- La précision dépend de la qualité de base de données.
- Temps de traitement plus long à cause de choix de modèle.
- Exigences matérielles plus élevées à cause de la complexité des algorithmes.
- Limitations de généralisation : Ils peuvent présenter des limites en termes d'adaptation à des situations inconnues ou à des populations spécifiques.
- La nécessité d'une expertise et des compétences spécialisées en analyse de données .

4.3 Etude comparatif entre les solutions existantes

Le tableau 2 présente une étude comparative entre les solutions existantes. Il permet la prise de décision quant au choix de la solution la plus appropriée pour répondre aux besoins du projet.

Tableau 2: Tableau comparatif entre les solutions existantes

	Les capteurs	**Les algorithmes d'apprentissage automatique**
Simplicité de système	Moyenne	Complexe
Degré de précision	Moins précis	Plus précis que les capteurs
Intégrité	Moyenne	Difficile
Expertise technique	Oui	Oui
Prix	Cher	Cher

Les technologies étudiées présentent des inconvénients tels que leur coût élevé, leur dépendance à des matériels spécifiques, le partage des données et des performances insatisfaisantes.
Pour ces raisons, Il est nécessaire de rechercher une solution simple, efficace et facile à intégrer dans le poste de pilotage de l'avion .

5 Solution proposée

5.1 Présentation :

Notre solution consiste à développer une application Android simple, conviviale et facile à utiliser pour la détection de la somnolence en utilisant un appareil mobile intégré dans le cockpit d'avion.

La caméra de l'appareil mobile est utilisée pour suivre les changements de comportement de l'utilisateur, tandis que l'analyse faciale est effectuée via ML Kit Face Detection API .

L'application alerte le pilote en cas de détection de problèmes classés en deux types :

Evénement de somnolence ou bâillement.

L'application permet aussi à l'utilisateur d'envoyer rapidement des messages d'alerte, de passer des appels et de partager instantanément la localisation en cas de situation critique. Cette fonctionnalité garantit l'obtention d'une assistance rapide en cas d'urgence.

La figure 15 présente le schéma général du projet qui sera développé dans les chapitres suivants :

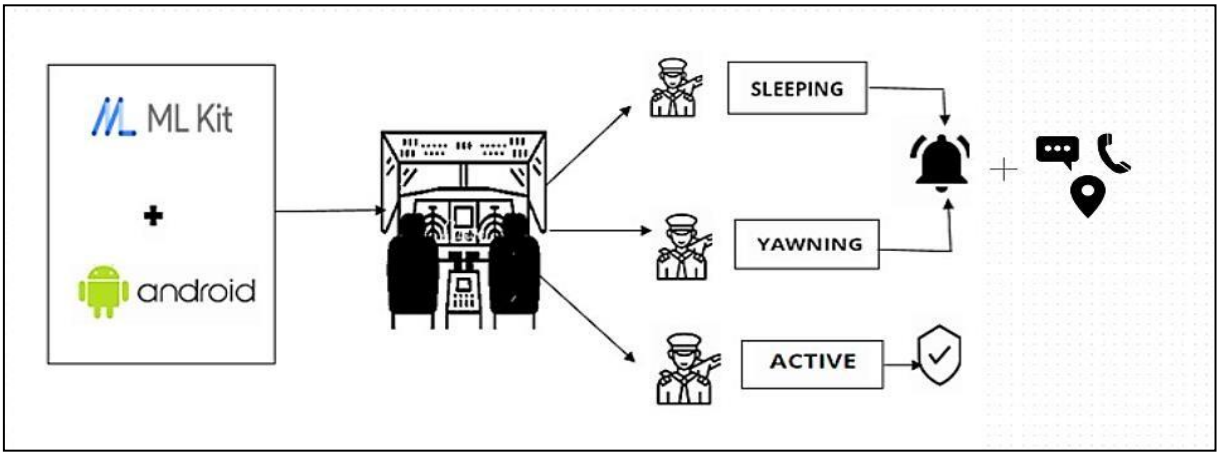


Figure 15: Schéma synoptique de la solution

5.2 Les caractéristiques

Les principales caractéristiques de notre projet sont les suivantes :

- Une application conviviale pour Android.
- La portabilité puisqu'elle sera disponible sur les téléphones Android.
- Pas d'utilisation de matériel externe inconfortable pour le conducteur.
- Fonctionnement hors ligne et en temps réel.

5.3 Originalité :

Pour mettre en évidence les originalités de notre solution par rapport aux systèmes basés sur des algorithmes d'apprentissage automatique étudiés dans la partie 4.2 nous présentons le tableau 3.

Tableau 3 : Etude comparative

	Application basée sur ML KIT et Google Play Services	**Système basé sur des algorithmes d'apprentissage automatique**
Développement personnalisé	• Prêt à l'emploi et facile à intégrer dans lesapplications. • Elle offre une interface simple pour accéder aux Fonctionnalités.	• Nécessite un développement personnalisé pour créer un modèlespécifique adapté aux besoins.
Entraînement des modèles	• Le processus d'entraînement des modèles est réalisé par Google.	• Il faut fournir des propres données d'entraînement et former les modèles.
Évolutivité	• Géré par Google, ce qui signifie qu'il est hautement évolutif et bénéficie de mises à jour régulières. • Les modèles d'apprentissage automatique sont Continuellement améliorés par Google .	• Les mises à jour doivent être gérées par les développeurs eux- mêmes.
Coût	• Les coûts sont en fonction de l'utilisation et des fonctionnalités utilisées.	• Les coûts sont é l ev é s en termes de temps et de matériel.

6 Planning du projet

Étant donné que le projet comprend plusieurs tâches à accomplir, il était nécessaire de répartir judicieusement les tâches tout au long de la période du stage afin de gérer efficacement le temps. ci-dessous la figure 16 de diagramme de Gantt qui illustre la répartition temporelle.



Figure 16: Diagramme du GANTT

7 Conclusion

Le premier chapitre de ce travail nous a permis de situer notre projet dans son contexte global. Nous avons d'abord présenté l'entreprise d'accueil, sa typologie de production ainsi que ses différents produits. Ensuite, nous avons exposé le cadre du projet en introduisant le sujet, en posant la problématique et en définissant les objectifs. Nous avons également réalisé une étude comparative des solutions existantes. De plus nous avons présenté une synthèse de la solution proposée à travers un schéma synoptique.

Le prochain chapitre sera dédié à l'analyse et spécification. On spécifiera l'étude fonctionnelle et la modélisation conceptuelle qui constituent notre système.

Chapitre 2 : ANALYSE & SPÉCIFICATION

1 Introduction

Pour notre solution, nous devons d'abord réaliser une analyse fonctionnelle approfondie qui comprend à la fois l'étude des besoins fonctionnels et non fonctionnels. Ensuite, nous procéderons à l'analyse de ces exigences en utilisant divers diagrammes adaptés. Enfin, nous réaliserons une modélisation conceptuelle pour représenter la solution.

2 Identification des besoins

L'identification des besoins est une étape cruciale dans le processus de développement d'un projet. Elle vise à comprendre en profondeur le contexte et l'objectif du système, ainsi qu'à déterminer les besoins fonctionnels et non fonctionnels du système.

2.1 Besoins fonctionnels

Les besoins fonctionnels sont les fonctionnalités spécifiques que le système doit offrir pour répondre aux exigences.

Notre projet doit offrir les fonctionnalités suivantes :

- L’application doit être capable de détecter la présence d'une personne.
- L'application doit utiliser l’analyse faciale.
- L'application doit être capable de surveiller et de détecter les signes de somnolence.
- L'application doit suivre en temps réel.
- L’application doit être en mesure d'émettre des alertes visuels et sonores.
- L’application doit envoyer un message d’alerte et un appel vocal automatiquement
- L’application doit fournir la localisation.

2.2 Besoins non fonctionnels

Les besoins non fonctionnels sont des critères qui évaluent le fonctionnement d'un système. Nous présenterons dans cette section les spécifiques que notre application doit respecter :

- ✓ Performance : L'application doit fournir des résultats rapides pour assurer une surveillance en temps réel efficace.

- ✓ Sécurité : Les données doivent être traitées de manière sécurisée et confidentielle en respectant les normes de protection des données personnelles.

- ✓ Fiabilité : L'application doit être fiable et fonctionne d'une manière cohérente même dans des conditions variables.

- ✓ Adaptabilité : L'application doit être capable de s'adapter à des différents environnements d'éclairage, angles de caméra et variations faciales pour assurer une détection précise dans diverses situations.

- ✓ Convivialité : L'interface utilisateur de l'application doit être conviviale et intuitive en permettant une utilisation facile sans distraction pour les pilotes.

- ✓ Intégration aisée : L'application doit être intégrable facilement pour une utilisation harmonieuse.

- ✓ Disponibilité : L'application doit être disponible et accessible en tout temps, afin d'assurer une surveillance continue.

- ✓ Maintenance : L'application doit être facilement maintenable, avec des mises à jour régulières pour améliorer les performances et ajouter de nouvelles fonctionnalités si nécessaire.

- ✓ Coût : L'application doit être économiquement viable.

Ces besoins non fonctionnels garantissent une utilisation sûre et efficace.

3 Analyse fonctionnelle

Dans cette section, nous utilisons les diagrammes de bêtes à cornes, de pieuvre et SWOT pour analyser et comprendre les besoins fonctionnels et non fonctionnels de notre solution.

3.1 Diagramme de Bête à cornes

Le diagramme de Bête à cornes illustré dans la figure 17 est une méthode d'analyse fonctionnelle utilisée dans les projets d'innovation et d'optimisation.

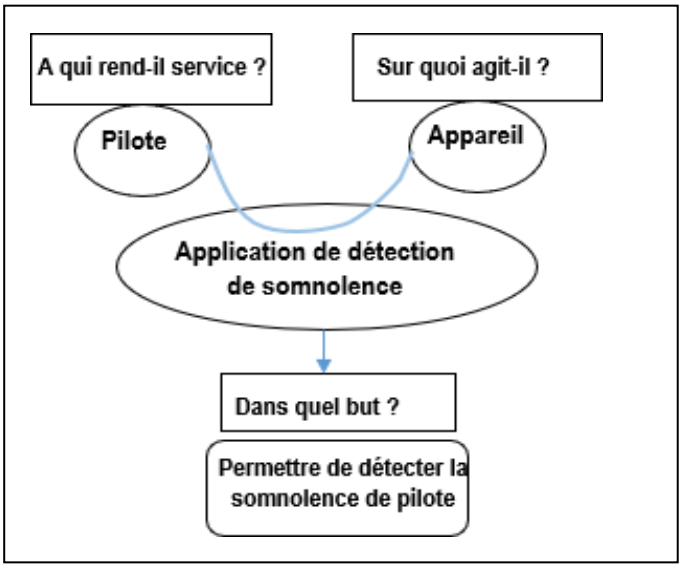


Figure 17 : Diagramme de Bête à cornes

3.2 Diagramme de Pieuvre

Le diagramme de pieuvre illustré en figure 18, est une représentation visuelle utilisée pour décrire les différentes parties pertinentes de l'application et leurs interactions.

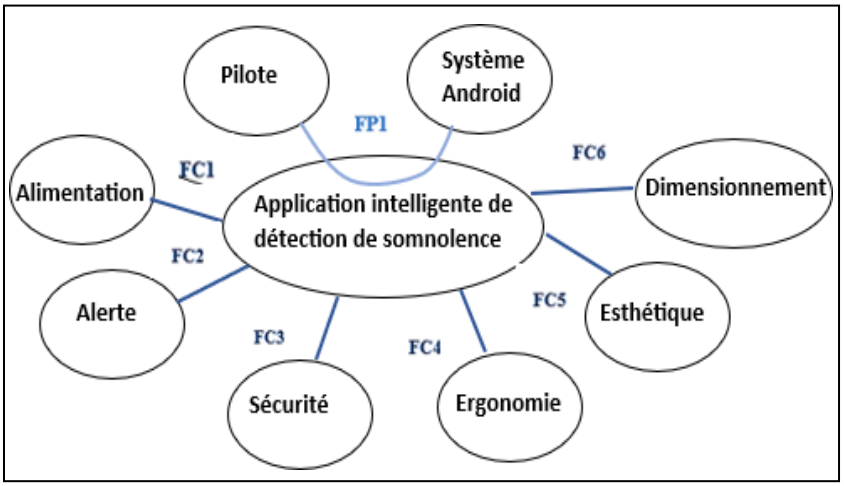


Figure 18 : Diagramme de pieuvres

La fonction principale ainsi que les fonctions contraintes sont expliquées plus en détails dans le tableau 4 :

Tableau 4 : Description des fonctions principales et contraintes du système

Fonction	Description
FP1	Détecter la somnolence de pilote à travers un appareil embarquant un système d'exploitation Android.
FC1	Assurer l'autonomie d'appareil.
FC2	Avoir une notification sonore perceptible.
FC3	Assurer la sécurité de pilote et l'aéronef.
FC4	Assurer une utilisation facile et performante.
FC5	Avoir une interface simple, conviviale et attrayante
FC6	L'appareil doit être peu encombrant dans le cockpit.

3.3 Diagramme SWOT

Le diagramme SWOT présenté dans la figure 19 est important pour une application, car il permet d'évaluer d'une manière approfondie les facteurs internes et externes qui peuvent influencer son développement, sa performance et son succès.

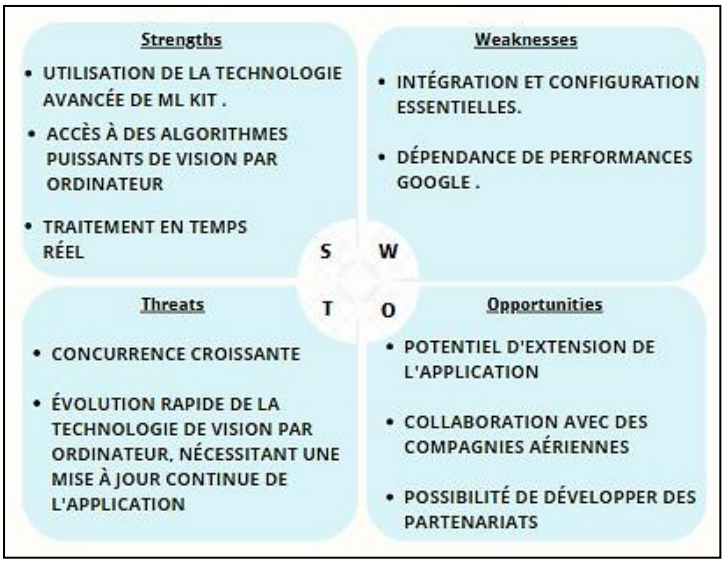


Figure 19 : Diagramme SWOT

4 Modélisation conceptuelle

La modélisation conceptuelle est une méthode utilisée pour représenter les concepts, les relations et les processus clés d'un système ou d'une application de manière abstraite et conceptuelle. Elle vise à capturer les concepts et les interactions essentiels sans entrer dans les détails de mise en œuvre. Dans cette partie on va utiliser la méthode conceptuelle UML.

4.1 Le Langage de Modélisation Unifié

Le langage UML est constitué de diagrammes intégrés, utilisés par les développeurs informatiques pour la représentation visuelle des objets, des états et des processus.
Le langage de modélisation peut servir de modèle pour un projet et garantir une architecture d'information structurée, il peut également aider les développeurs à présenter leur description d'un système d'une manière compréhensible pour les spécialistes externes.

4.2 Diagramme de cas d'utilisation

Le diagramme présenté dans la figure 20 permet de visualiser les interactions principales entre l'utilisateur et l'application.

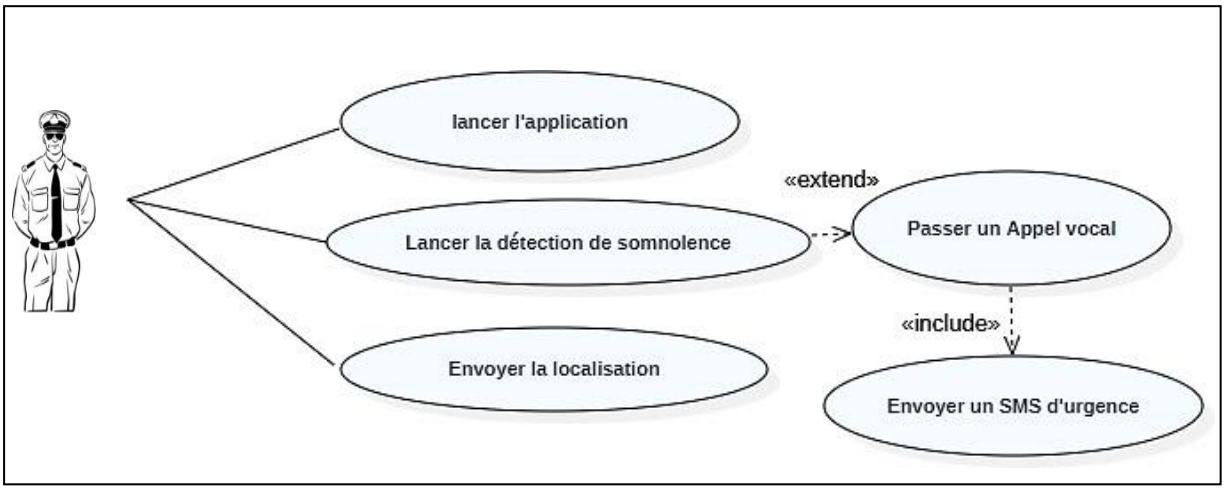


Figure 20 : Diagramme de cas d'utilisation globale

Le cas d'utilisation « Lancer l'application » décrit le processus de démarrage et de l'application. Une description textuelle de ce cas d'utilisation est illustrée dans le tableau 5.

Tableau 5: Description de cas d'utilisation « Lancer l'application »

Nom : « Lancer application »
Acteur : Pilote
Précondition : L'utilisateur doit installer et configurer l'application sur son appareil.
Scenario nominal : 1. L'utilisateur appuie sur l'icône de l'application pour la lancer. 2. L'application démarre et affiche l'écran d'accueil.
Post-conditions : L'application est lancée avec succès. L'utilisateur a accès aux fonctionnalités.

Le cas d'utilisation « lancer la détection de somnolence » concerne le processus de lancement de l'opération principale. Une description textuelle de ce cas d'utilisation est illustrée dans le tableau 6.

Tableau 6 : Description de cas d'utilisation « Lancer la détection de somnolence »

Nom : « Lancer la détection de somnolence »
Acteur: Pilote
Précondition : L'application doit avoir l'autorisation d'accéder au camera de l'appareil. L'opération de détection a été initiée avec succès.
Scenario nominal : 1. L'utilisateur accède à la fonctionnalité de détection de somnolence. 2. L'utilisateur autorise la caméra. 3. L'utilisateur click sur le bouton start. 4. L'utilisateur démarre l'opération.
Post-condition : L'opération est terminée avec succès.

Le cas d'utilisation « Envoyer la localisation » concerne le processus de transmission de la localisation géographique à un contact désiré. Une description textuelle de ce cas d'utilisation est illustrée dans le tableau 7.

Tableau 7 : Description de cas d'utilisation « Envoyer la localisation »

Nom : « Envoyer la localisation »
Acteur : Pilote
Pré-condition : - L'application doit avoir l'autorisation d'accès à la localisation. - L'application doit être en mesure de récupérer avec précision les coordonnées de localisation de l'appareil.
Scenario nominal : 1. L'utilisateur ouvre l'application sur son appareil mobile. 2. L'application demande l'autorisation de localisation. 3. L'application récupère les coordonnées de localisation de l'appareil. 4. L'utilisateur sélectionne le numéro d'envoi de sa localisation. 5. L'application envoie la localisation et les informations associées à la destination prévue.
Post-conditions : La localisation de l'utilisateur a été envoyée avec succès au destinataire.

4.3 Diagramme de séquences

Ce diagramme est considéré comme l'un des diagrammes UML les plus intéressant, car il facilite la compréhension du fonctionnement du système. Il représente d'une manière séquentielle et chronologique les interactions entre les objets du système, ainsi que les interactions avec les éléments externes. Ce diagramme est présenté dans la figure 21.

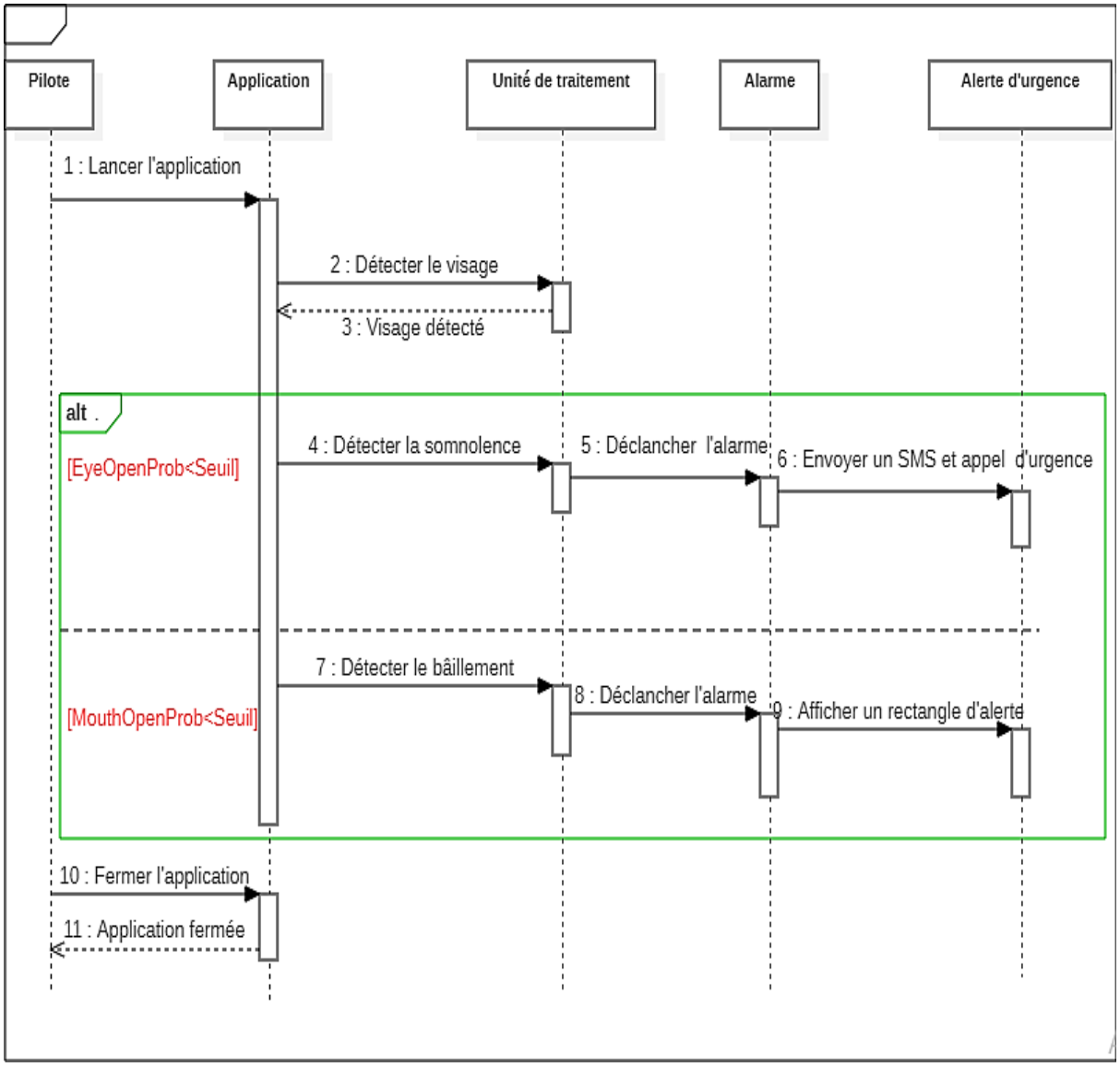


Figure 21 : Diagramme de séquences

4.4 Diagramme d'activité :

Un diagramme d'activité est un type de diagramme UML qui représente le flux d'activités ou de processus à l'intérieur d'un système. Il est utilisé pour modéliser le comportement dynamique d'un système, en mettant l'accent sur les actions, les décisions et les flux de contrôle.

La figure 22 décrit une séquence d'instructions et des décisions de notre application.

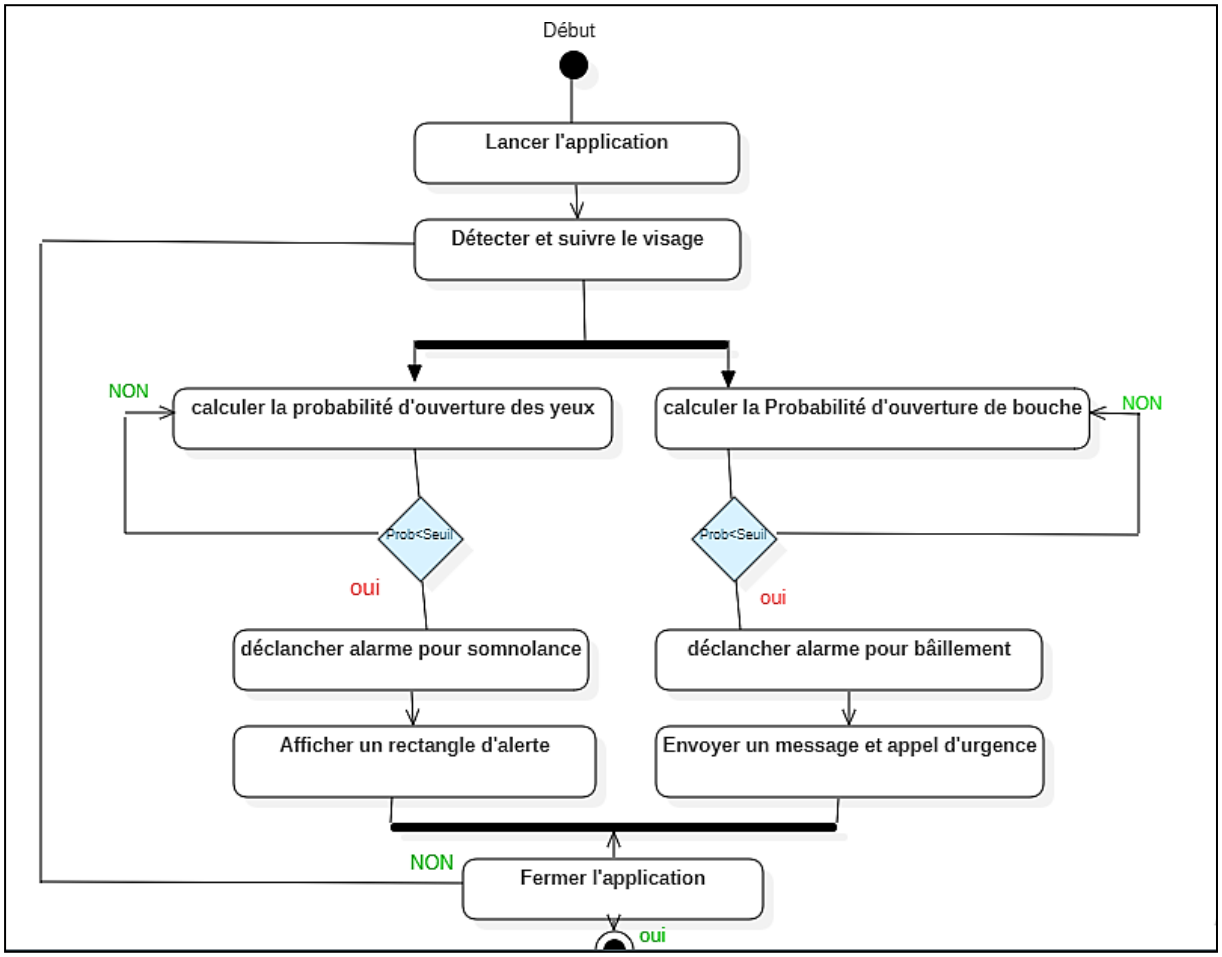


Figure 22 : Diagramme d'activité

5 Conclusion :

Nous avons entamé le deuxième chapitre en définissant les exigences fonctionnelles et non fonctionnelles que notre solution devra satisfaire à l'aide de plusieurs diagrammes.

Dans le prochain chapitre, nous explorerons l'environnement de travail et l'implémentation d'application.

Chapitre 3 : Développement & Implémentation

1 Introduction

Ce chapitre permet de fournir une vue d'ensemble du contexte dans lequel l'application va évoluer et des outils qui seront utilisés pour sa création.
Nous commençons dans la première partie par présenter l'environnement matériel et logiciel, ainsi que les choix technologiques sur lesquels l'application a été développée. Ensuite, dans la deuxième partie, nous décrivons en détail le processus d'implémentation de l'application.

2 Environnement matériel

L'implémentation et la réalisation de notre système sont fortement influencées par l'environnement matériel :

- Pour le développement et la réalisation, nous utilisons une station de travail « PC » avec les caractéristiques suivantes :
  - Processeur : Intel ®, Core™ i5 - 7200U CPU 2.5GHz.
  - 4Go de mémoire vive.
  - Système d'exploitation : Windows 10 professionnel.

- Pour l'émulation et le débogage nous utilisons un smartphone Android « Xiaomi Redmi 12C PRO » avec les caractéristiques suivantes [10] :
  - Écran : Ecran TFT LCD de 6,71 pouces + Résolution HD de 720 x 1650 pixels.
  - Performance : Vitesse d'horloge maximale de 1,8 GHz, couplée à 4 Go de RAM.
  - Stockage: 128 Go de stockage interne.
  - Caméras : Caméra arrière 50 MP et caméra frontale de 5 MP.
  - Batterie : Capacité de 5000 mAh.
  - Connectivité : 4G LTE, Wi-Fi 802.11 b/g/n, Bluetooth 4.2 et GPS.

3 Environnement logiciel

Dans cette section, nous décrivons l'environnement logiciel sur lequel nous avons travaillé. Pour le développement de notre application nous avons opté pour l'utilisation de :

3.1 Android studio

3.1.1 Présentation

Android Studio [11] est l'IDE officiel pour le développement d'applications Android, basé sur l'outil puissant d'édition de code IntelliJ IDEA. Il propose de nombreuses fonctionnalités pour améliorant la productivité lors de la création des applications Android. Voici quelques-unes de ces fonctionnalités illustrées dans le tableau 8 :

Tableau 8 : Les fonctionnalités d'Android studio

Fonctionnalités	Description
Éditeur de code avancé	Android Studio offre un éditeur de code intelligent avec la saisie automatique, la refactorisation, la recherche rapide et la détection d'erreurs en temps réel.
Interface de conception conviviale	Android Studio comprend un éditeur de mise en page graphique qui permet de créer et personnaliser l'interface utilisateur.
Gestion de projet simplifiée	Android Studio facilite la création, l'organisation et la gestion des fichiers de projet Android.
Profilage et optimisation des performances	Android Studio comprend des outils de profilage avancés pour analyser les performances de l'application.

3.1.2 Version :

Tant que le système d'exploitation open source Android OS évolue de manière rapide et continue, le tableau 9 illustre les versions majeures d'Android avec leurs fonctionnalités clés :

Tableau 9 : Les versions d'Android

Version	Fonctionnalités
Android 1.0 (Alpha)	Offre Les fonctionnalités de base.
Android 2.0 (Éclair)	Intègre la navigation vocale.
Android 3.0 (Honeycomb)	Offre une navigation améliorée.
Android 4.0 (Sandwich)	Intègre la reconnaissance faciale.
Android 5.0 (Lollipop)	Prise en charge des appareils Android TV.
Android 6.0 (Marshmallow)	Lecteur d'empreintes digitales.
Android 7.0(Nougat)	Intègre Google Assistant.
Android 8.0 (Oreo)	Optimise les points d'accès Wi-Fi.
Android 9 (Pie)	Améliore la sécurité.
Android 10	Intègre le mode sombre
Android 11	Offre une messagerie plus pratique.
Android 12	Offre un Nouveau design.

La figure 23 represente l'evolution des version android au fil du temps.



Figure 23 : Evolution d'Android *[12]*

3.1.3 Architecture

L'architecture d'Android [13] est essentielle car elle offre une structure organisée pour le développement d'applications. Elle offre une séparation claire et facilite la maintenance et l'évolutivité des applications. Elle est basée sur un modèle à 5 couches comme illustrée dans la figure 24 :

- Application : Les applications Android sont développées et exécutées dans cette couche et chaque application fonctionne dans son propre processus.

- Framework d'application : Cette couche fournit un ensemble des composants et des services que les développeurs utilisent pour créer des applications Android.

- Bibliothèques système : Les bibliothèques système incluent des bibliothèques pour la gestion des bases de données, la gestion des graphiques et la gestion des sécurités.

- Android Runtime : Cette couche s'agit de l'environnement d'exécution des applications Android.

- Noyau Linux : Le noyau Linux fournit les services de base du système d'exploitation tels que la gestion des processus, la gestion de la mémoire et la gestion des matériels.

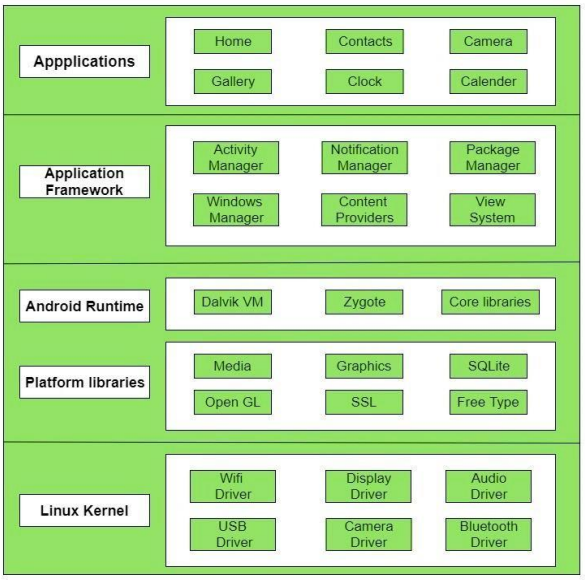


Figure 24 : Architecture Android

3.1.4 Structure du projet

La structure du projet Android [14] illustrée dans la figure 25 suit généralement une organisation en couches, comprenant les éléments suivants :

- Répertoire app : C'est le module principal de l'application, qui contient le code source de l'application, les ressources, les activités et les fragments.

- Répertoire AndroidManifest.xml : C'est le fichier qui définit les informations essentielles, telles que les permissions et les activités.

- Répertoire java : Ce répertoire contient les packages et les classes Java.

- Répertoire de ressource : Ce répertoire contient les fichiers XML, les fichiers de chaînes de caractères, les fichiers d'images et les fichiers de styles.

- Répertoire Gradle : Ce répertoire offre la gestion des dépendances et la compilation.

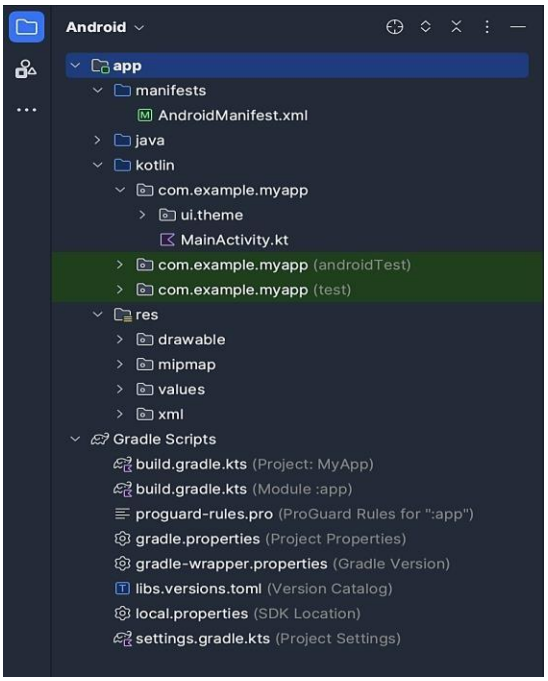


Figure 25 : Fichiers de projet de la vue Android *[14]*

Cette structure du projet permet d'avoir une organisation claire et modulaire du code pour faciliter le développement, la maintenance et la collaboration entre les développeurs.

3.2 Android JDK

Le JDK est un ensemble des outils et des bibliothèques essentiels pour le développement d'application Android avec le langage de programmation Java. Il intègre le compilateur Java, l'environnement d'exécution Java, et des bibliothèques spécifiques à Android. Grâce à ces fonctionnalités, les développeurs peuvent créer des applications robustes avec des capacités étendues de la plateforme et une expérience utilisateur de qualité. La figure 26 présente la version d'Android JDK.



Figure 26: Version de JDK

3.3 Android SDK

Le SDK Android est un ensemble des outils et des ressources fournis par Google pour le développement des applications. Il comprend des composants tels que les API, les émulateurs, les outils de développement et la documentation. Grâce à ces ressources, les développeurs peuvent créer des applications compatibles avec les différentes versions d'Android et exploiter les fonctionnalités de la plateforme avec une expérience utilisateur optimale. La figure 27 illustre les fonctionnalités dans Android SDK.

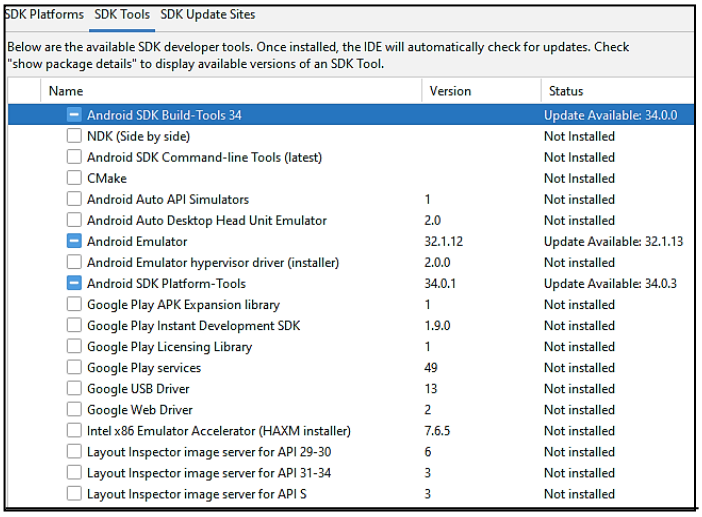


Figure 27 : Fonctionnalités d'Android SDK

4 Choix du Langage

Pour le développement de notre application, nous avons opté pour l'utilisation des langages de programmation suivants :

4.1 Java

Java [15] est un langage de programmation largement utilisé dans le développement mobile, en particulier pour la création d'applications Android. Il offre une portabilité élevée ce qui permet de réutiliser ou d'adapter facilement le code pour différentes plateformes. Il bénéficie d'une vaste bibliothèque standard. Il est apprécié pour sa performance, sa stabilité et son écosystème solide. En utilisant Java, les développeurs peuvent créer des applications mobiles puissantes, réactives et compatibles avec une grande variété de dispositifs

.



Figure 28: Logo de Java *[15]*

4.2 XML

XML [16] est un langage utilisé pour définir la structure de l'interface utilisateur, les ressources et les configurations de l'application. Il est utilisé pour créer des fichiers de mise en page, décrire les ressources (textes, couleurs et dimensions), spécifier les autorisations dans le fichier de manifeste et définir des animations et des transitions.

XML joue un rôle central dans la conception et la personnalisation de l'interface utilisateur des applications, offrant une approche claire et lisible pour organiser et décrire les éléments visuels et les données de l'application.



Figure 29 : Logo XML *[16]*

5 Framework et technologie

Dans le cadre du processus de développement de notre application, nous avons utilisé les technologies et Framework suivants :

5.1 ML Kit

ML Kit [17] est un Framework d'apprentissage automatique développé par Google qui permet aux développeurs d'ajouter des fonctionnalités d'intelligence artificielle à leurs applications mobiles Android et iOS d'une manière simple. Il offre des APIs prêtes à l'emploi pour la détection de visage, la reconnaissance de texte, la lecture de codes-barres et l'identification de la langue. Il offre les avantages ci-après :

- Facilité d'utilisation : Avec une interface simple et conviviale, les développeurs peuvent facilement implémenter des fonctionnalités d'intelligence artificielle sans nécessiter d'une expertise approfondie en apprentissage automatique.
- Modèles pré-entrainés : ML Kit fournit des modèles d'apprentissage automatique pré-entrainés, permettant aux développeurs de bénéficier rapidement des dernières technologies en IA sans avoir à concevoir et entraîner leurs propres modèles.
- Traitement local et sur le cloud : ML Kit offre la flexibilité de choisir entre le traitement local sur l'appareil ou l'utilisation des services cloud de Google, selon les besoins de l'application et les contraintes de ressources.

5.2 ML KIT Face Detection API

L'API de détection de visage de ML Kit [18] permet d'intégrer des modèles d'apprentissage automatique de détection de visage. Ces principales fonctionnalités sont les suivantes :

- Reconnaissance et localisation des traits du visage, tels que les yeux, les oreilles, les joues, le nez et la bouche.
- Obtention des contours précis des traits du visage, y compris des yeux, des sourcils, des lèvres et du nez.
- Identification des expressions faciales.
- Suivi et traitement en temps réel des visages sur les images et les vidéos.

5.3 Google Play Services

Google Play Services [19] est une suite des services fournis par Google pour les appareils. Il s'agit d'une couche logicielle intégrée dans le système d'exploitation Android, qui offre des fonctionnalités et des services essentiels aux utilisateurs et aux développeurs d'applications.
Ces services permettent notamment de :

- Bénéficier de mises à jour régulières pour améliorer la sécurité et les performances.
- Intégrer facilement des fonctionnalités telles que la localisation, les notifications push et l'authentification des utilisateurs.
- Assurer une compatibilité avec différents appareils Android et offrir une expérience utilisateur cohérente.
- Simplifier le développement d'applications en fournissant des API et des outils pour les développeurs.

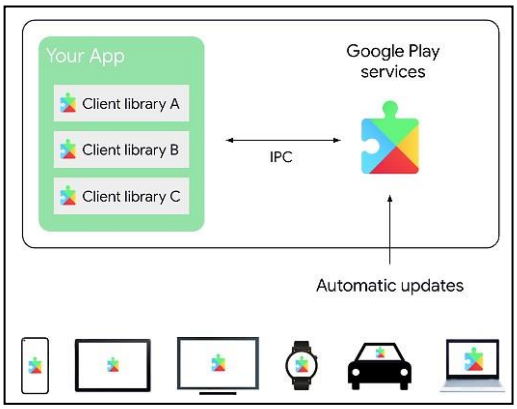


Figure 30: Mises à jour d'application *[19]*

6 Implémentation

Dans cette partie, nous allons mettre en œuvre les étapes nécessaires pour l'implémentation de l'application. Cela comprend la configuration initiale de l'environnement de développement, la gestion des dépendances et l'implémentation des fonctionnalités principales.

6.1 Configuration

Dans le développement du modèle, il est essentiel de prendre en compte les configurations des chemins d'installation et l'emplacement de modèle.

6.1.1 Chemins d'installation de modèle

Les modèles dans l'API de détection faciale [20] peut être installée de deux manières :

- Installation dégroupée : Les modèles sont téléchargés via les services Google Play.
- Installation groupée (Lot) : Les modèles sont associés de manière statique à l'application lors de sa compilation.

Le tableau 10 présente les principales différences entre les options d'installation.

Tableau 10 : Les différences entre options d'installation *[20]*

	Dégroupé	Lot
Où les modèles sont-ils stockés ?	Résident dans les services Google Play et n'est pas comptabilisé dans l'espace de stockage utilisé par cette appli	Enregistré dans un espace de stockage spécifique à l'application après l'installation.
Quel est l'impact de la taille du modèle sur la taille de l'application ?	Ne contribue pas à la taille de l'appli	Contribution directe à la taille de l'application
Quand les modèles sont-ils mis à jour ?	Mise à jour automatique dès la publication d'une version plus récente	Mettre à jour l'application pour mettre à jour le modèle
Quand les modèles sont-ils téléchargés ?	Vous devez télécharger les modèles avant de les utiliser	Tous les modèles et fonctionnalités sont inclus lors de l'installation de l'application, ils sont donc utilisables immédiatement.
Quand les modèles sont-ils supprimés de l'appareil ?	Les services Google Play ne suppriment le modèle de l'espace de stockage que si toutes les applications qui en dépendent sont désinstallées	Les modèles sont supprimés de l'espace de stockage spécifique à l'application lorsque cette dernière est désinstallée.

Le tableau 11 présente les caractéristiques distinctives entre les options d'installation

Tableau 11 : Les caractéristique des options d'installations *[21]*

Caractéristique	Sans bundles	Regroupées
Implémentation	Le modèle est téléchargé de manière dynamique via les services Google Play.	Le modèle est associé à votre application de manière statique au moment de la compilation.
Taille d'application	Augmentation d'environ 800 ko.	Augmentation d'environ 6,9 Mo.
Délai d'initialisation	Vous devrez peut-être attendre que le modèle soit téléchargé pour la première utilisation.	Le modèle est disponible immédiatement

Pour optimiser notre application, nous avons choisi d'utiliser le chemin d'installation dégroupé avec Google Play Services. Cette décision vise à réduire la taille de l'application, simplifier la gestion des mises à jour et minimiser l'impact sur la mémoire RAM des appareils.

6.1.2 Installation de modèle

Pour installer le modèle de l'API de détection de visage avec l'option dégroupée, on ajoute la déclaration spécifiée dans la figure 31 au fichier AndroidManifest.xml de l'application.

```
<application ...>
      ...
      <meta-data
          android:name="com.google.mlkit.vision.DEPENDENCIES"
          android:value="face" >
      <!-- To use multiple models: android:value="face,model2,model3" -->
</application>
```

Figure 31: Déclaration de modèle *[21]*

Pour utiliser le modèle avec les services Google Play, on ajoute la dépendance des bibliothèques ML Kit, telle qu'indiquée dans la figure 32, au fichier Gradle de l'application.

```
dependencies {
  // ...
  // Use this dependency to use the dynamically downloaded model in Google Play Services
  implementation 'com.google.android.gms:play-services-mlkit-face-detection:17.1.0'
}
```

Figure 32: Dépendance des bibliothèques ML kit *[21]*

6.2 Méthodologie

Notre système global est composé de deux parties essentielles illustrés dans la figure 33 : La détection de la somnolence pour prévenir les risques liés à la fatigue et la gestion des situations d'urgence pour assurer une réponse rapide et appropriée.

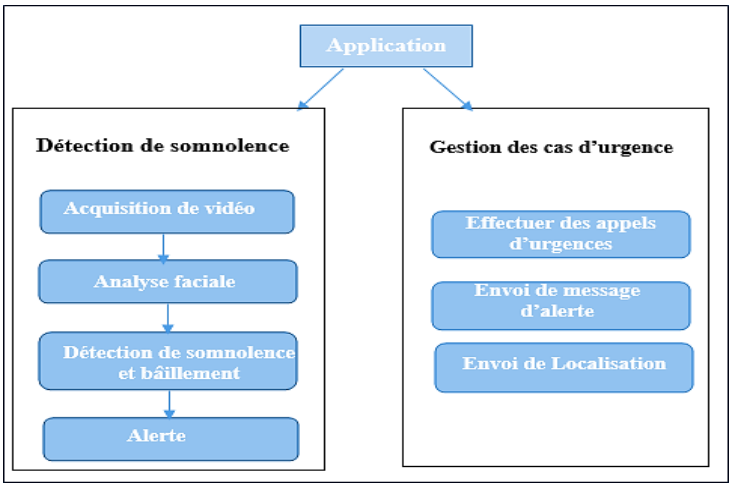


Figure 33 : Architecture de système globale

6.2.1 Détection de somnolence

Cette partie du système est entièrement dédiée à la détection de la somnolence. Elle comprend les étapes qui permettent de surveiller et d'analyser l'état de pilote en temps réel.

Etape 1 : Acquisition de vidéo

L'acquisition du flux vidéo en temps réel du visage nécessite l'utilisation de la caméra embarquée dans l'appareil pour capturer les images du visage. Ces images seront ensuite envoyées au processeur de ML Kit pour analyser les expressions faciales. Il est nécessaire d'une part d'activer la permission qui permet à l'application d'accéder à la caméra de l'appareil et d'autre part d'importer les classes suivantes :

Tableau 12 : Les classes d'affichage

Classe	Rôle
Camera Source Preview	Cette classe est utilisée pour afficher l'aperçu de la caméra dans une interface.
Graphic Overlay	Cette classe est utilisée pour superposer des éléments graphiques.

Etape 2 : Analyse faciale

L'API de ML Kit possède la capacité de détecter les visages dans les images ou dans les vidéos. Elle prend en entrée une image ou une séquence vidéo et en retour elle fournit une liste des visages détectés. Voici quelques concepts essentiels utilisés pour assurer la détection précise :

1- **Suivi des visages** : Le suivi continu des visages facilite l'analyse et le traitement des visages tout au long de la séquence vidéo, offrant ainsi une meilleure compréhension des expressions faciales de la personne détectée.

2- **Identification des points de repère** : L'API de détection des visages offre la fonctionnalité de localisation des points de repère sur un visage détecté. Un point de repère représente un point d'intérêt à l'intérieur du visage. Dans notre application nous utilisons les points de repère suivants : L'œil gauche, l'œil droit, la bouche droite, la bouche gauche, le bout de la bouche et la base du nez.

3- **La classification** : La classification dans ML Kit permet de déterminer si certaines caractéristiques du visage sont présentes. Actuellement, ML Kit prend en charge deux classifications : Les yeux ouverts et le sourire.

Pour assurer la détection des visages à l'aide de ML Kit, il est nécessaire d'implémenter les classes illustrées dans le tableau 13 :

Tableau 13 : Classes de détection de visage

Classe	Rôle
Vision.face.Face	Cette classe représente les informations dans un visage détecté.
Vision.face.FaceDetectorOptions	Cette classe représente les options de configuration du détecteur de visage.
vision.face.FaceLandmark	Cette classe représente les points de repère sur un visage détecté.

Etape 3 : Détection de somnolence et bâillement.

ML KIT renvoie une valeur de probabilité d'ouverture des yeux sous forme décimale, comprise entre 0 et 1. Dans cette échelle, une valeur de 0 représente généralement un œil fermé, tandis qu'une valeur de 1 représente un œil complètement ouvert. De plus elle renvoie une valeur de probabilité de sourire sous forme décimale comprise entre 0 et 1. La probabilité d'ouverture de bouche est calculée en utilisant les points de repère du visage détecté. Les seuils montrés dans le tableau 14 sont choisis après la réalisation des plusieurs tests.

Tableau 14 : Conditions & Evènement

Condition	Evènement
(L'œil gauche est fermé et la probabilité d'ouverture de l'œil) < 0.1	L'œil gauche est fermé
(L'œil droite est fermé et la probabilité d'ouverture de l'œil) < 0.1	L'œil droite est fermé
L'œil gauche est fermé et L'œil droite est fermé	Alerte de somnolence
Bouche ouverte et degré d'ouverture < 90	Bouche ouverte
Bouche ouverte et la probabilité de sourire <0.4	Alerte de bâillement

Etape 4 : Alerte

Le système d'alerte est conçu pour détecter la somnolence et les bâillements de pilote. En cas de détection de somnolence, un mécanisme d'alarme se déclenche après « 10 secondes » de vérification des conditions, en ce moment le système envoie automatiquement un appel à un contact d'urgence afin de solliciter de l'aide ou d'avertir des situations dangereuses, tout en envoyant un message d'urgence. Une assistance vocale est également mise en place pour rappeler le pilote de rester vigilant.

Lorsqu'un bâillement est détecté, une alarme est déclenchée après « 5 secondes » de vérification des conditions, accompagnée d'un affichage visuel sous la forme d'un rectangle rouge dans le but d'attirer l'attention du pilote et de lui faire prendre conscience de la situation critique. Le système lance un message vocal pour que le pilote soit prudent.

6.2.2 Gestion des cas d'urgence :

Cette partie du système est programmée pour envoyer automatiquement un appel au contact approprié, accompagné d'un SMS d'urgence. De plus, le pilote a la possibilité d'envoyer sa localisation, comprenant les informations sur son altitude, sa latitude, sa longitude, sa vitesse et son adresse complète. Avant d'utiliser les fonctionnalités de gestion des cas d'urgence, il est nécessaire d'activer les permissions illustrées dans le tableau 15 :

Tableau 15 : Liste des permissions

Permission	Rôle
CALL_PHONE	Cette autorisation permet de passer des appels téléphoniques depuis l'appareil.
ACCESS_FINE_LOCATION	Cette autorisation est utilisée pour fournir des fonctionnalités basées sur la localisation.
SEND_SMS	Cette autorisation permet d'envoyer des messages SMS à partir de l'appareil.

Afin de garantir l'envoi des messages il est nécessaire d'implémenter le package suivant, ainsi que sa classe associée.

- **Le package Android. Telephony** : C'est une partie du Framework Android qui fournit des classes pour interagir avec les fonctionnalités téléphoniques.
- **La classe Android. Telephony. SmsManager** : Cette classe permet de gérer l'envoi et la réception de messages SMS.

Pour assurer une localisation précise, il est nécessaire d'implémenter le package suivant, ainsi que ses classes associées.

- **Le Package Android. Location** : C'est une partie du Framework Android qui fournit des classes et des interfaces pour travailler avec les services de localisation.
- **La classe Android.location.Address** : Cette classe représente une adresse géographique, telle que le pays et la ville.
- **La classe Android.location.Geocoder** : Cette classe permet de convertir les coordonnées de la localisation en une adresse lisible par l'homme.
- **La classe Android.location.Location** : Cette classe représente les informations de localisation telles que la latitude, la longitude et l'altitude.

6.3 Installation

À partir d'Android 11 ou d'une version ultérieure, il est possible de déployer et de déboguer l'application sans fil depuis l'ordinateur. Pour utiliser le débogage sans fil [22] , on associe l'appareil au poste de travail en utilisant un code QR d'association. Cela permet d'établir une connexion sécurisée qui simplifie le processus de développement et de test sans avoir à manipuler des câbles physiques. Pour se connecter on doit suivre les étapes suivantes :

1. Ouvrir Android Studio et sélectionner l'option "Pair Devices Using Wi-Fi" : La boîte de dialogue "Pair devices " s'affiche comme illustré dans la figure 34.



Figure 34 : Boite de dialogue

2. Activer les options pour les développeurs sur l'appareil Android.
3. Activer le débogage via le Wi-Fi
4. Sélectionner "Pair device with QR code" et scanner le code QR.

7 Conclusion

Dans ce chapitre, nous avons fourni une description de l'environnement matériel et logiciel de notre projet. Ensuite, nous avons également mentionné les langages de programmation et les technologies utilisés pour la réalisation de notre application. Enfin, nous avons décrit le schéma d'implémentation de notre application et nous avons abordé les différentes étapes employées tout au long du processus de développement. Ce chapitre nous a permis de donner un aperçu complet de l'environnement, des outils et des technologies utilisés, ainsi que des démarches suivies pour la mise en place de notre application. Le prochain chapitre sera dédié aux résultats, aux tests et à l'analyse des performances de l'application.

Chapitre 4 : Test et Evaluation

1 Introduction

Après avoir présenté les procédures de développement et d'implémentation de notre application, nous nous concentrons dans ce chapitre sur les résultats obtenus et l'évaluation de notre application. Tout d'abord nous commençons par la présentation des interfaces de notre application, puis nous procédons à des tests approfondis. Enfin, nous analysons les performances de l'application, mettant en évidence ses points forts.

2 Résultat

Dans cette section, nous introduisons l'architecture de notre application et les interfaces qui la composent. Cette présentation sert à l'évaluation ultérieure de la performance de notre application.

2.1 Structure hiérarchique

Notre application présente une structure hiérarchique comme illustrée dans la figure 35. Chaque élément de l'arborescence correspond à une partie spécifique de l'application, permettant un accès rapide aux fonctionnalités principales et aux sous-fonctionnalités associées.

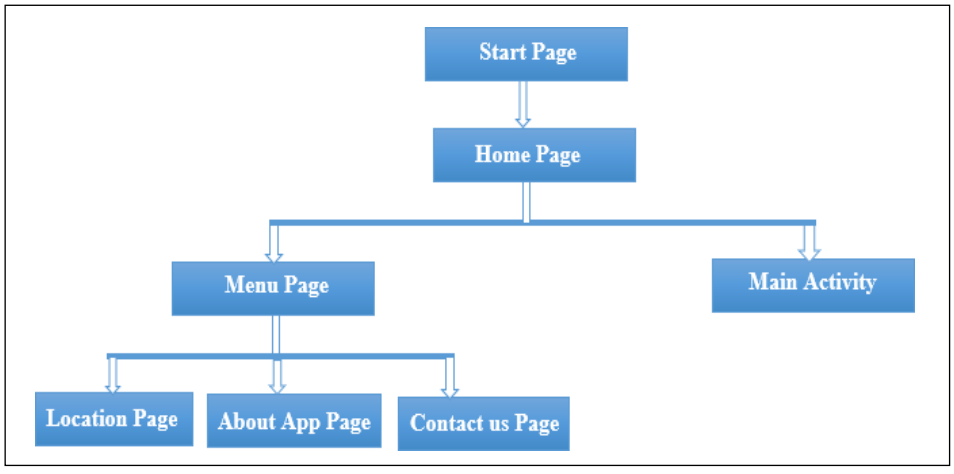


Figure 35 : Arborescence d'application

2.2 Interfaces

L'interface utilisateur est la partie qui permet aux utilisateurs d'interagir avec le système. C'est la couche visible et interactive du logiciel comprenant des éléments tels que des boutons, des menus, des formulaires et des fenêtres. L'objectif de l'interface utilisateur est de faciliter l'utilisation d'application en rendant les fonctionnalités accessibles.

Dans cette section, nous allons présenter une série de captures d'écran illustrant les différentes fonctionnalités que nous avons développées dans notre projet, afin de valider notre travail.

2.2.1 Logo et Dénomination

Le nom "Smart Cockpit for Aircraft" a été choisi pour notre application afin de souligner son rôle essentiel dans la création d'une expérience de vol sécurisée et immersive.

Le logo d'application montré dans la figure 36 est représenté par un œil et un avion. Ce choix symbolise la fonction principale de détection de somnolence.

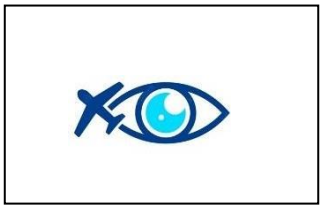


Figure 36 : Logo d'application

2.2.2 Start Page

La figure 37 présente la première interface qui s'affiche lorsque l'utilisateur ouvre l'application.



Figure 37 *:* Start Page

2.2.3 Home Page

Cette interface est illustrée dans la figure 38, elle présente deux boutons principaux :

Bouton "Let's Start" : Une fois l'utilisateur appuie sur ce bouton, il est redirigé vers l'interface Où il peut commencer à utiliser les fonctionnalités de détection de somnolence.

Bouton "Emergency" : dédié aux situations d'urgence. En le pressant, l'application réalise automatiquement un appel à un contact prédéfini. Cette fonctionnalité est essentielle pour fournir une assistance rapide et un niveau de sécurité supplémentaire.



Figure 38 : Home Page

2.2.4 Main Activity Page

Cette interface est conçue d'une manière minimaliste pour éviter toute distraction et mettre l'accent sur le suivi du visage. Les éléments visuels sont simples afin de ne pas surcharger l'interface et de permettre une expérience utilisateur claire.
Dans cette section, il y a trois situations différentes : la situation normale, la situation de somnolence et la situation de bâillement. Chacune de ces situations représente un état spécifique dans lequel une personne peut se trouver.

- **La situation normale :**

Dans ce cas, le suivi du visage se déroule normalement et sans problème comme montré dans la figure 39. L'application peut détecter et analyser les mouvements du visage de l'utilisateur pour évaluer son état d'éveil et de vigilance.

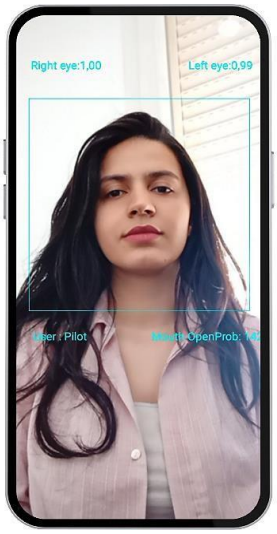


Figure 39 : Situation normale

- **La situation de somnolence**

Dans la situation de somnolence illustrée dans la figure 40 : si les yeux restent fermés pendant 10 secondes, une alerte est déclenchée. En conséquence, un appel d'urgence est automatiquement effectué, accompagné d'un message d'urgence préenregistré.



Figure 40 : Situation de somnolence

- **La situation de bâillement**

Dans la situation de bâillement présentée dans la figure 41 : lorsque la bouche reste ouverte pendant 5 secondes, une alerte sonore est déclenchée avec une alerte visuelle sous forme de rectangle rouge pour attirer l'attention de l'utilisateur sur son état de fatigue.

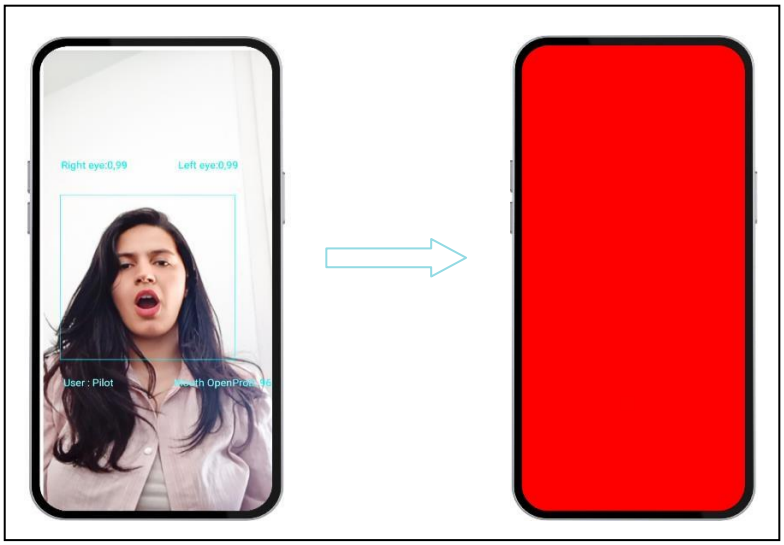


Figure 41 : Situation de bâillement

2.2.5 Menu Page

La page de menu de l'application comporte quatre fragments offrant des fonctionnalités spécifiques tels que le retour à la page d'accueil, la localisation, informations sur l'application et la possibilité de contacter la société. Cette conception permet à l'utilisateur de naviguer facilement dans l'application et d'accéder aux fonctionnalités principales et supplémentaires ou d'obtenir de l'aide si nécessaire. Cette interface est montrée dans la figure 42.



Figure 42 : Menu Page

2.2.6 Location Page

La page de localisation montrée dans la figure 43 affiche les informations de la localisation, telles que l'altitude, la longitude, la latitude, la vitesse, la région et la position de l'utilisateur. Cette page comporte également un bouton « Send Location » qui permet d'envoyer un SMS contenant les informations de localisation automatiquement au destinataire prédéfini en simple clique.



Figure 43: Location page

2.2.7 About App Page

L'interface montrée dans la figure 44 regroupe les principales fonctionnalités et informations de l'application en un seul endroit. Elle offre aux utilisateurs un aperçu global des services offerts par l'application.



Figure 44 : About app page

2.2.8 Contact Us Page

Les utilisateurs peuvent trouver les informations de contact de la société « AVIONAV » tels que l'adresse e-mail, LinkedIn, le numéro de téléphone et l'adresse dans l'interface présentée dans la figure 45.



Figure 45 : Contact Us Page

3 Test

Dans cette partie, nous procédons à l'évaluation de la précision et des performances de notre système. Pour évaluer la précision, nous utiliserons un test manuel. Quant à l'évaluation des performances, nous utilisons le profileur d'Android Studio.

3.1 Précision

Nous avons réalisé des tests manuels dans différents scénarios avec un groupe de 20 personnes. L'objectif était de vérifier la précision de notre système. Nous avons pris soin de sélectionner un groupe diversifié de participants, couvrant différentes tranches d'âge et sexes. Les tests ont été effectués dans des environnements variés.

3.1.1 Test avec luminosité variée

Dans ce cas, nous avons effectué des tests comme illustrés dans la figure 46 pour évaluer la capacité de détection de la somnolence dans des conditions d'éclairage différentes, notamment un éclairage fort, moyen et faible.

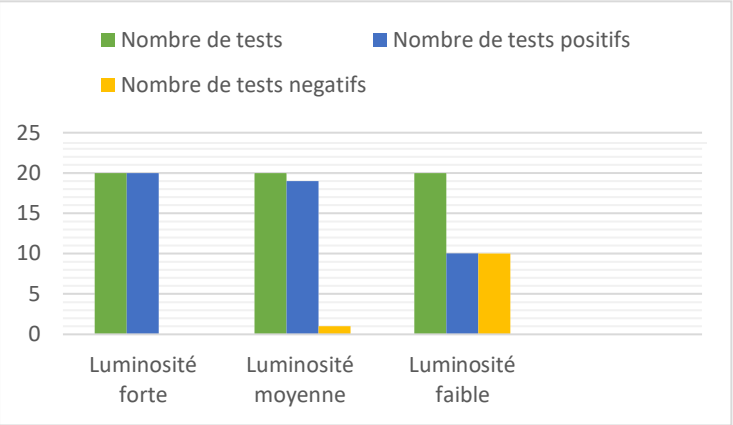


Figure 46 : Test de précision avec différents types de luminosité

Nous avons obtenu un taux de précision de 82%. En se basant sur ce résultat, il est recommandé d'utiliser l'application dans des milieux où l'éclairage est suffisamment bon pour obtenir une précision élevée.

3.1.2 Test avec une inclinaison de caméra

Dans la figure 47 nous avons effectué des tests supplémentaires sur notre application en utilisant des inclinaisons différentes de caméra.

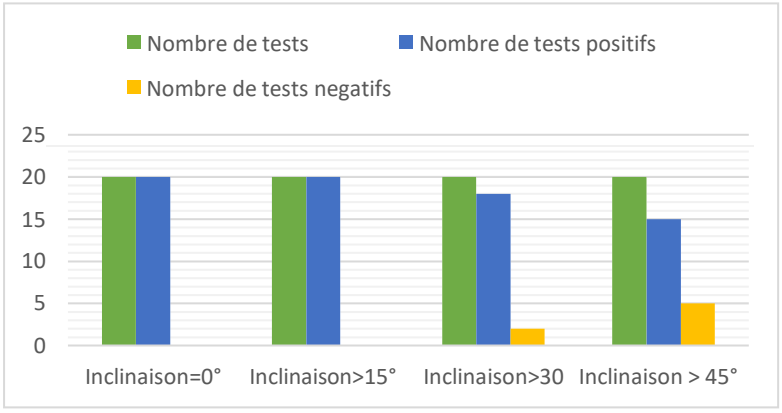


Figure 47 : Test de précision avec des inclinaisons de caméra.

Dans ce cas, nous avons observé une précision de 91% en utilisant des inclinaisons différentes. Cependant, pour une utilisation parfaite de l'application, il est recommandé de positionner l'appareil directement en face de l'utilisateur.

3.1.3 Test divers

Nous allons procéder à des tests présentés dans les figures 48 et 49 en tenant compte des différentes conditions telles que le port de bavette, le port de lunettes de vue et le port de lunettes solaires.

- **Cas 1 : Détection de somnolence**

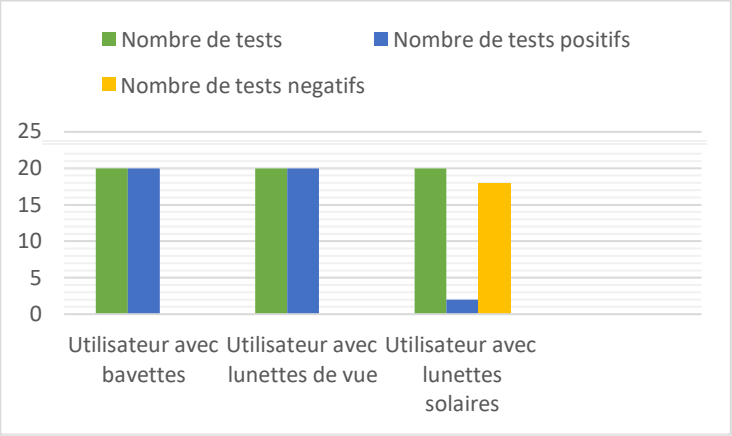


Figure 48 : Test de précision -cas 1-

Avec une précision de l'application de 70% dans ce cas, les tests ont démontré que l'application présente une précision plus élevée lorsqu'elle est utilisée sans lunettes solaires.

- **Cas 2 : Détection de bâillement**

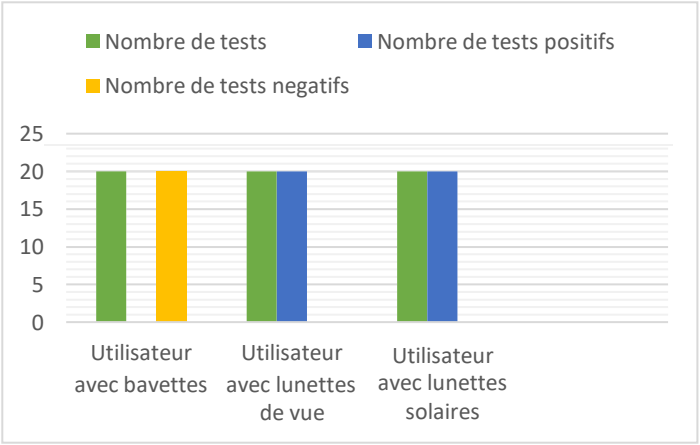


Figure 49 : Test de précision -Cas 2-

Dans ce cas, nous avons obtenu une précision de 67%. Les tests ont révélé que l'utilisation d'un isolant en face de la bouche entraîne une précision moins fiable et affecte la visibilité des mouvements faciaux.

3.2 Performance

Dans cette partie, nous utilisons le profileur d'Android Studio pour évaluer les performances de l'application. Cet outil permet d'analyser les différentes métriques de performance, telles que le temps d'exécution, la consommation de mémoire et l'utilisation du processeur.

3.2.1 Profileur

Le profileur [23] est un outil intégré dans l'environnement de développement Android Studio qui permet aux développeurs d'analyser les performances et le comportement d'une application Android pendant son exécution. Il offre des fonctionnalités avancées pour diagnostiquer et résoudre les problèmes de performances. Il contient 3 types :

- **Le Profileur de processeur** : est un service qui permet de surveiller et d'analyser en temps réel l'utilisation du processeur lors de l'exécution d'une application. Il identifie les problèmes de performances tels que les ralentissements ou les blocages.
- **Le Profileur de mémoire** : permet de surveiller l'allocation et la libération de la mémoire, ainsi que de détecter les fuites de mémoire potentielles.
- **Le Profileur d'énergie** : est un service qui surveille la consommation d'énergie de l'application. Il identifie les zones de l'application qui consomment une quantité excessive d'énergie, ce qui permet aux développeurs de prendre des mesures pour optimiser l'efficacité énergétique de l'application.

3.2.2 Etude des performances

Dans cette section, nous avons évalué les performances d'application en utilisant le profileur d'Android Studio et en analysant les performances des activités et fragments de l'application. Lorsque on utilise l'application, le profileur commence à collecter des données sur l'utilisation du CPU, de la mémoire et de l'énergie et il surveille en temps réel ces métriques. La figure 50 est une séquence d'activités et de fragments dans l'application y compris «Menu activity" , ainsi que les fragments "Home", "Contact us " et "About App"

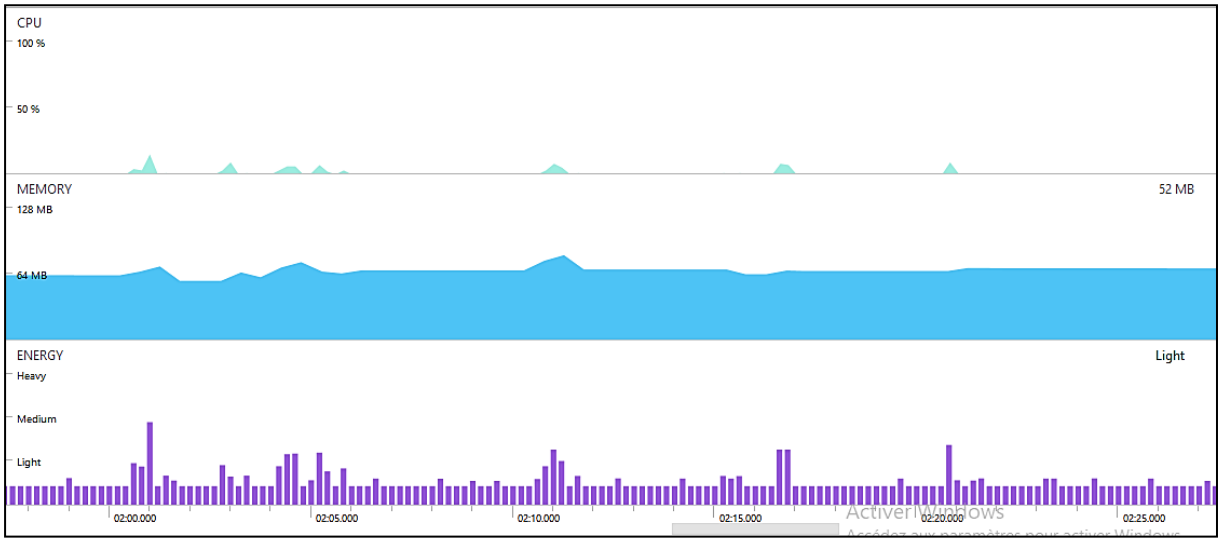


Figure 50 : Vue chronologique des activités et fragments

D'après cette figure, les performances de l'application ont été analysées comme suit :

- Utilisation du CPU : La faible variation de Pourcentage de CPU indique que l'application est bien optimisée et n'exécute pas de tâches intensives en arrière-plan.

- Consommation de mémoire : Une consommation de mémoire de 52 MB indique que l'application utilise une quantité modérée de mémoire.

- Consommation d'énergie "Light" : Cette indication signifie que l'application n'a pas un impact significatif sur la batterie de l'appareil et elle utilise une quantité minimale d'énergie pendant l'exécution.

Pour une évaluation plus complète des performances, on teste l'application dans l'activité principale de détection de somnolence. La figure 51 représente l'activité principale de l'application lorsque le signal de détection de sommeil est activé.

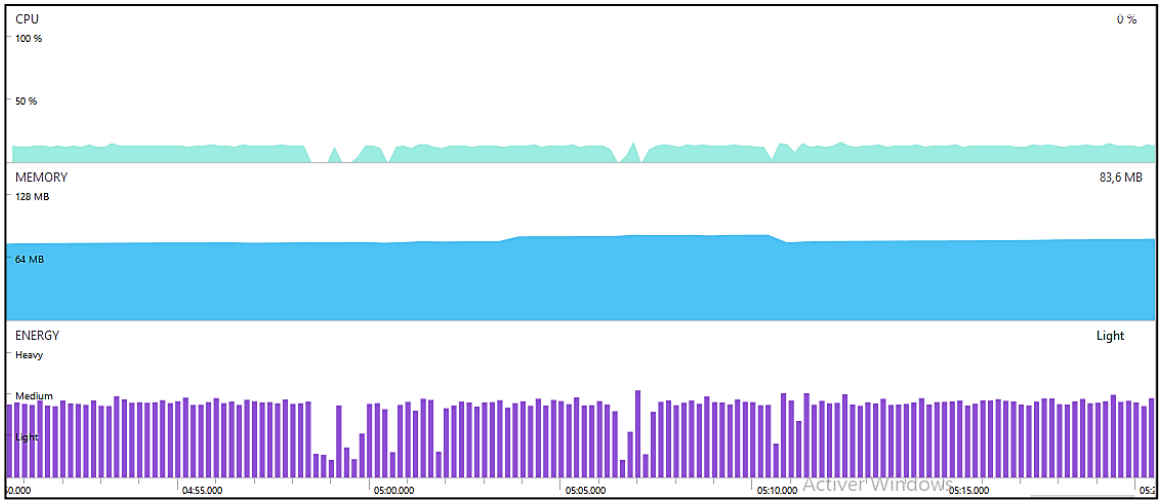


Figure 51 : Vue des performances d'activité principale

D'après la figure 53 , les performances de l'application ont été analysées comme suit :

- Utilisation du CPU : Une variation d'utilisation du CPU légère d'environ 14 % indique que l'application exécute des tâches avec une puissance de traitement optimisée.

- Consommation de mémoire : Une consommation de mémoire de 83,6 Mo indique que l'application utilise une quantité modérée de mémoire. L'ajout de fonctionnalités comme la détection de sommeil et l'analyse du visage nécessitent un stockage de données supplémentaires en mémoire.

- Consommation d'énergie : Une consommation d'énergie proche du niveau "Medium" indique que l'application utilise une quantité raisonnable d'énergie pendant son exécution. L'alarme peut augmenter légèrement la consommation d'énergie, car elle nécessite des ressources pour déclencher les sons .

D'après ces deux figures, on constate que l'application présente des performances élevées : L'utilisation du CPU est légère, la consommation de mémoire est modérée et la consommation d'énergie est maîtrisée.

4 Conclusion

Dans ce chapitre, nous avons procédé à une analyse approfondie des interfaces de notre application en examinant leur conception et leur convivialité. Nous avons également réalisé différents tests pour évaluer la précision et le bon fonctionnement de l'application dans des scénarios variés, en assurant qu'elle répond aux besoins des utilisateurs de manière fiable. De plus, nous avons utilisé le profileur d'Android Studio pour évaluer les performances de l'application, en analysant des métriques telles que l'utilisation du CPU, la consommation de mémoire et l'utilisation de l'énergie.

Conclusion générale

Face à l'évolution du domaine de l'aéronautique, la sécurité est devenue une préoccupation majeure. Les risques de sécurité évoluent constamment et l'un des principaux problèmes auxquels l'industrie aéronautique est confrontée est la somnolence des pilotes.
La somnolence des pilotes peut entraîner des conséquences graves sur la sécurité des vols, ce qui augmente les risques d'erreurs humaines pouvant entraîner des accidents aériens.
Devant ce défi, il est crucial de développer des systèmes de sécurité qui peuvent détecter la somnolence des pilotes et prendre des mesures appropriées pour prévenir les accidents. C'est là qu'intervient notre application intelligente.

Dans le manuscrit élaboré, nous avons détaillé les différentes procédures de conception de l'application.

Nous avons débuté dans le 1er chapitre par une analyse approfondie des contraintes spécifiques au domaine de l'aéronautique, en mettant en évidence les risques associés à la somnolence des pilotes. Ensuite, nous avons examiné différentes solutions existantes afin de définir notre approche. Puis nous avons fourni une vision globale de notre solution avec schéma synoptique.

Dans le 2ème chapitre du manuscrit, nous avons effectué une analyse approfondie des besoins de notre projet. Cette analyse a porté à la fois sur les besoins fonctionnels et les besoins non fonctionnels. Pour les besoins fonctionnels, nous avons identifié les principales fonctionnalités que l'application doit offrir. Pour présenter ces besoins de manière claire et concise, nous avons utilisé des diagrammes, tels que le diagramme de cas d'utilisation, le diagramme de séquence et le diagramme d'activité. L'analyse approfondie des besoins fonctionnels et non fonctionnels nous a permis de cerner les exigences de l'application et d'orienter les étapes vers le développement.et l'implémentation.

Le 3ème chapitre de notre manuscrit se concentre sur les outils utilisés pour développer l'application. Nous commencerons par présenter les environnements matériels et logiciels requis. Ensuite, nous discuterons des langages de programmation, des Framework et des technologies que nous avons utilisés pour faciliter le processus de développement. Enfin, nous détaillerons la procédure d'implémentation, en mettant en évidence les étapes clés.

Le 4ème chapitre de notre manuscrit se focalise sur les tests et l'évaluation de notre application. D'abord, nous avons réalisé une analyse approfondie des interfaces pour assurer une convivialité optimale. Ensuite, nous avons effectué une série de tests exhaustifs afin d'évaluer la précision et le bon fonctionnement de l'application dans divers scénarios. Enfin, une évaluation des performances de l'application a également été réalisée par le profileur.

En conclusion, notre application présente plusieurs caractéristiques avantageuses. Elle s'adapte à des autres secteurs, elle est facile à intégrer dans le poste de pilotage, abordable en termes de coût et peut fonctionner sans connexion. De plus, elle démontre de bonnes performances en termes d'utilisation du CPU, de la mémoire et de l'énergie, ce qui garantit une expérience fluide tout en préservant les ressources ainsi que l'intégration de ML Kit et de Google Play Services offre une fonctionnalité impressionnante et enrichissante à notre application.

Dans une perspective d'amélioration, nous souhaitons accroître la précision de notre solution en intégrant une caméra nocturne ou infrarouge. Cette évolution vise à renforcer la fiabilité des résultats obtenus. En offrant une meilleure capacité à capturer des images dans des conditions de faible luminosité ou d'obscurité, notre solution pourra fournir des informations plus précises et répondre aux attentes élevées de nos utilisateurs. L'ajout de cette fonctionnalité représente une amélioration significative pour assurer des performances optimales.

Référence

[1] Linkedin de société: https://www.linkedin.com/company/avionav/.consulté le 01/02/2023

[2] Avionav site web : https://avionav.net/. consulté le 01/02/2023

[3] La presse « 40 avions légers tunisiens exportés vers les marchés étrangers» , publié le 13/01/2020

[4] Netherlands Aerospace Centre «Independent research on Pilot Fatigue » , publié le 01/01/2019

[5] Marie Bohacova « Methodology of short fatigue crack detection by the eddy current method in a multi-layered metal aircraft structure» 35, 2013,597-608.

[6] Ministére de la transition écologique «ETUDE DE LA FATIGUE EN AERONAUTIQUE » DOC AA 395/98.

[7] Capteur EEG : https://mystidia.com/roumani_s-physiology-9/. consulté le 10/03/2023.

[8] Capteur EMG : https://www.gotronic.fr/art-detecteur-emg-grove-101020058-21514.htm. consulté le 10/03/2023.

[9] Capteur de mouvement https://evasommeil.com/25-la-polygraphie-ventilatoire-pv. consulté le 10/03/2023.

[10] Fiche technique Redmi 12C: https://specificationsplus.com/fr/redmi-12c/. consulté le 01/06/2023.

[11] Découvrir Android Studio : https://developer.android.com/studio/intro?hl=fr. consulté le 20/03/2023.

[12] The Evolution of Android : https://www.cellphonedeal.com/blog/the-evolution-of-android. consulté le 21/03/2023.

[13] Guide de l'architecture: https://developer.android.com/topic/architecture?hl=fr. consulté le 21/03/2023.

[14] Structure de projet: https://developer.android.com/studio/intro?hl=fr. consulté le 24/03/2023.

[15] Introduction Java: https://www.java.com/fr/. consulté le 15/03/2023.

[16] Introduction XML: https://fr.wikipedia.org/wiki/Extensible_Markup_Language. consulté le 15/03/2023

[17] Lars Birkedal Nick Rothwell Mads Tofte David N. Turner «The ML Kit Version 1»119, March 17, 1993.

[18] Shweta Sahu, Mansi Sahu, Anjali Choudhary, Sanjay Kashyap, Soniya Wadhwa «TEXT, FACE AND OBJECT DETECTION BY USING MACHINE LERNING» 7, 2582-5208.

[19] Services Google Play: https://developers.google.com/android/guides/overview?hl=fr. consulté le 01/04/2023].

[20] Chemins d'installation des modèles de ML Kit sur Android: Chemins d'installation des modèles de ML Kit sur Android. consulté le 10/04/2023.

[21] Détecter des visages avec ML Kit sur Android: https://developers.google.com/ml-kit/vision/face-detection/android?hl=fr.. consulté le 15/03/2023.

[22] Exécuter des applications sur un appareil: https://developer.android.com/studio/run/device?hl=fr.consulté le15/04/2013.

[23] Profiler les performances de votre application: https://developer.android.com/studio/profile?hl=fr. consulté le 01/05/2023.

[24] Les étapes d'apprentissage automatique: https://fr.wikipedia.org/wiki/Apprentissage_automatique.consulté le 15/03/2023

[25] Les versions d'android: https://fr.wikipedia.org/wiki/Historique_des_versions_d%27Android.consulté le 01/06/2023.